



清华大学计算机科学与技术系本科专业主修课

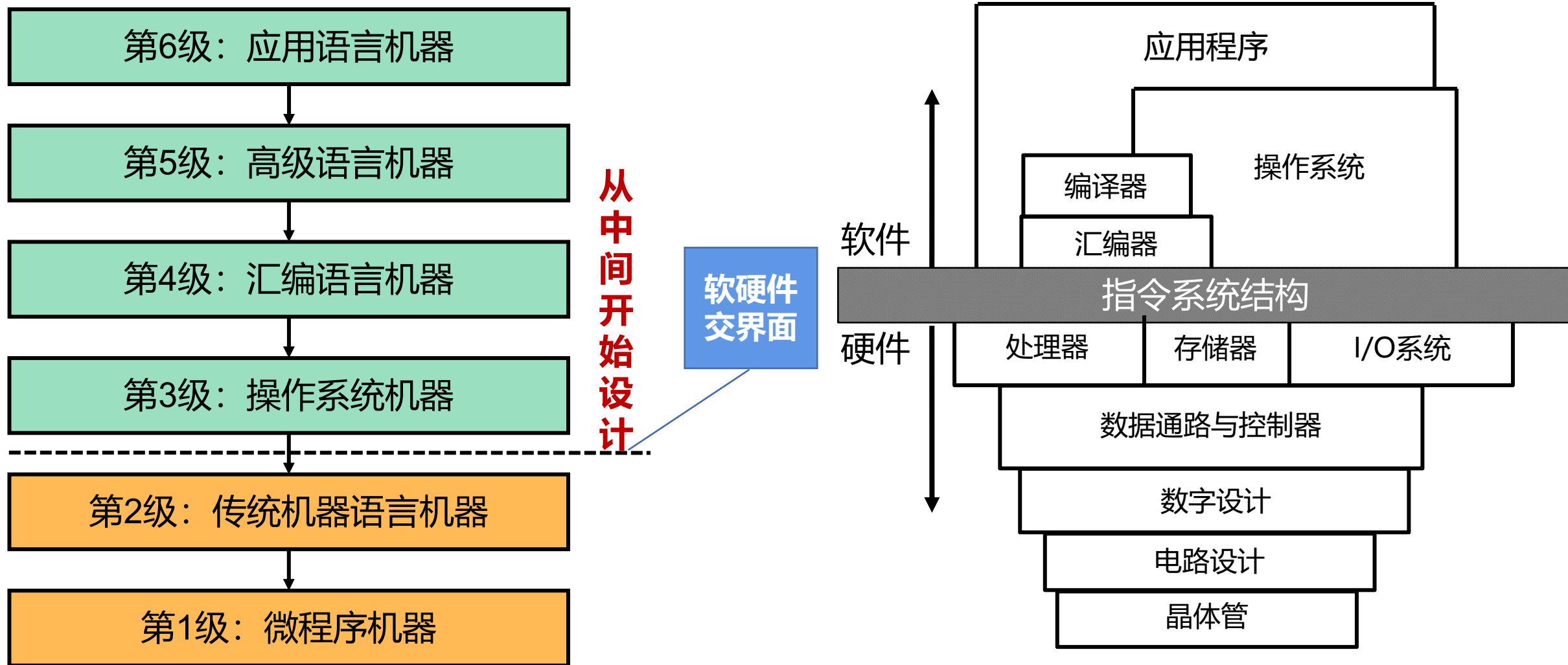
计算机系统结构

第2章 指令系统的设计

2026年春季学期

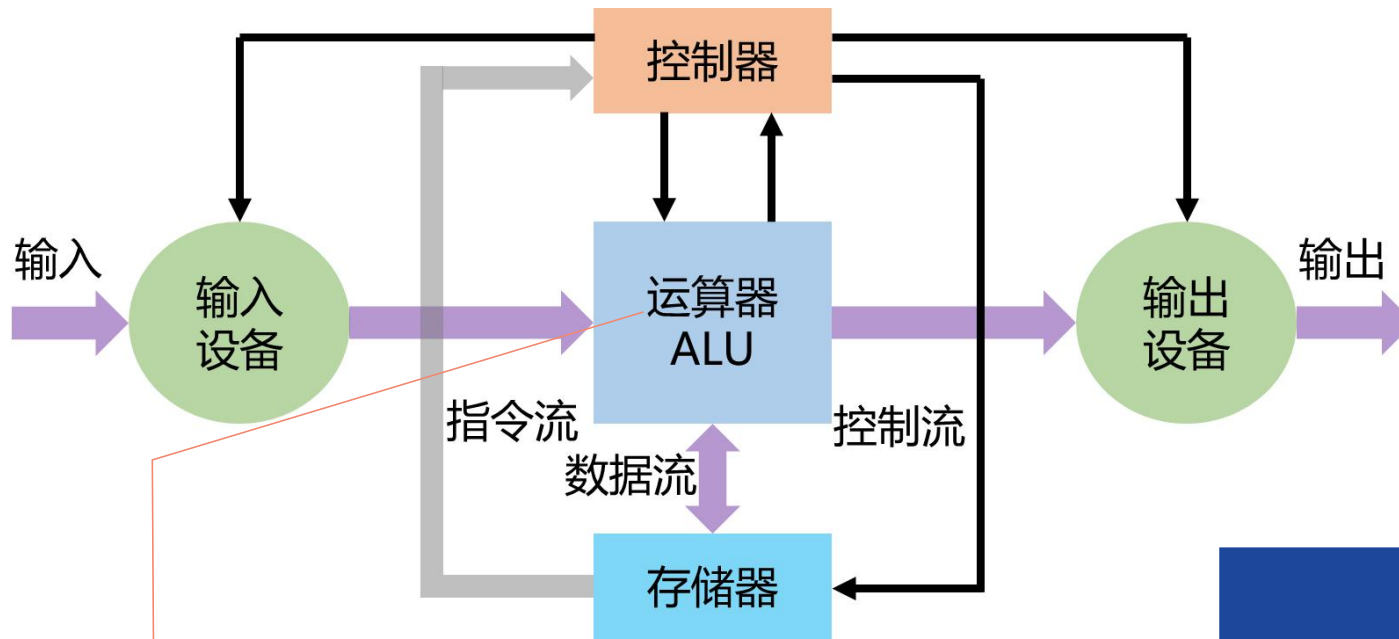
从指令系统开始

第 2 页



冯·诺依曼架构

第 3 页



加法、乘法、除法、平方根、求
余数、向量加、向量乘等



冯·诺依曼
John von Neuman
著名匈牙利裔美籍
数学家、计算机科学
家、物理学家和
化学家

存储程序工作原理

- 将计算过程描述为由许多条指令按一定顺序组成的程序，并放入存储器保存
- 指令按其在存储器中存放的顺序执行；
- 由控制器控制整个程序 and 数据的存取以及程序的执行。

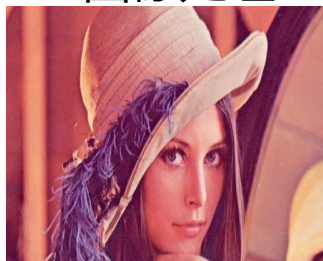
回顾指令系统

第 4 页

科学计算



图像处理



游戏娱乐



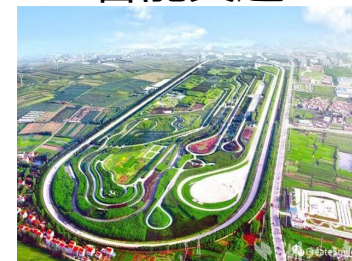
智能家居



智慧城市



智能交通



计算机

- 指令就是指挥处理器正确运行的指示和命令；
每条指令定义处理器的一定的功能
- 指令集是指一个处理器能够执行的所有指令。
- 程序是一系列按一定顺序排列的指令，程序执行过程就是处理器的工作过程。

一般指令由两部分组成：

操作码	操作数（地址码）
-----	----------

如何设计？

从一个简单实例开始

第 5 页

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

运算公式： $A+B=C$

操作分解分析

实例1的要求：

- 处理器一次只能完成1次算术运算
- 设计要求：
 - ◆ 指令集定义
 - ◆ 执行序列分析

输入值	$A_{1,1}$	$A_{1,2}$	$A_{2,1}$	$A_{2,2}$
输入值	$B_{1,1}$	$B_{1,2}$	$B_{2,1}$	$B_{2,2}$
计算	$C_{1,1}$	$C_{1,2}$	$C_{2,1}$	$C_{2,2}$
输出	$C_{1,1}$	$C_{1,2}$	$C_{2,1}$	$C_{2,2}$

■三个指令

◆数字传输类指令

□ 输入数据 A

load R_A, A

□ 输出数据 B

store R_B, B

◆运算类指令

□ 两个数相加 $A+B=C$

add R_A, R_B, R_C

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \quad A+B=C$$

load $R_0, A_{1,1}$

load $R_1, A_{1,2}$

load $R_2, A_{2,1}$

load $R_3, A_{2,2}$

load $R_4, B_{1,1}$

load $R_5, B_{1,2}$

load $R_6, B_{2,1}$

load $R_7, B_{2,2}$

add R_0, R_4, R_8

add R_1, R_5, R_9

add R_2, R_6, R_{10}

add R_3, R_7, R_{11}

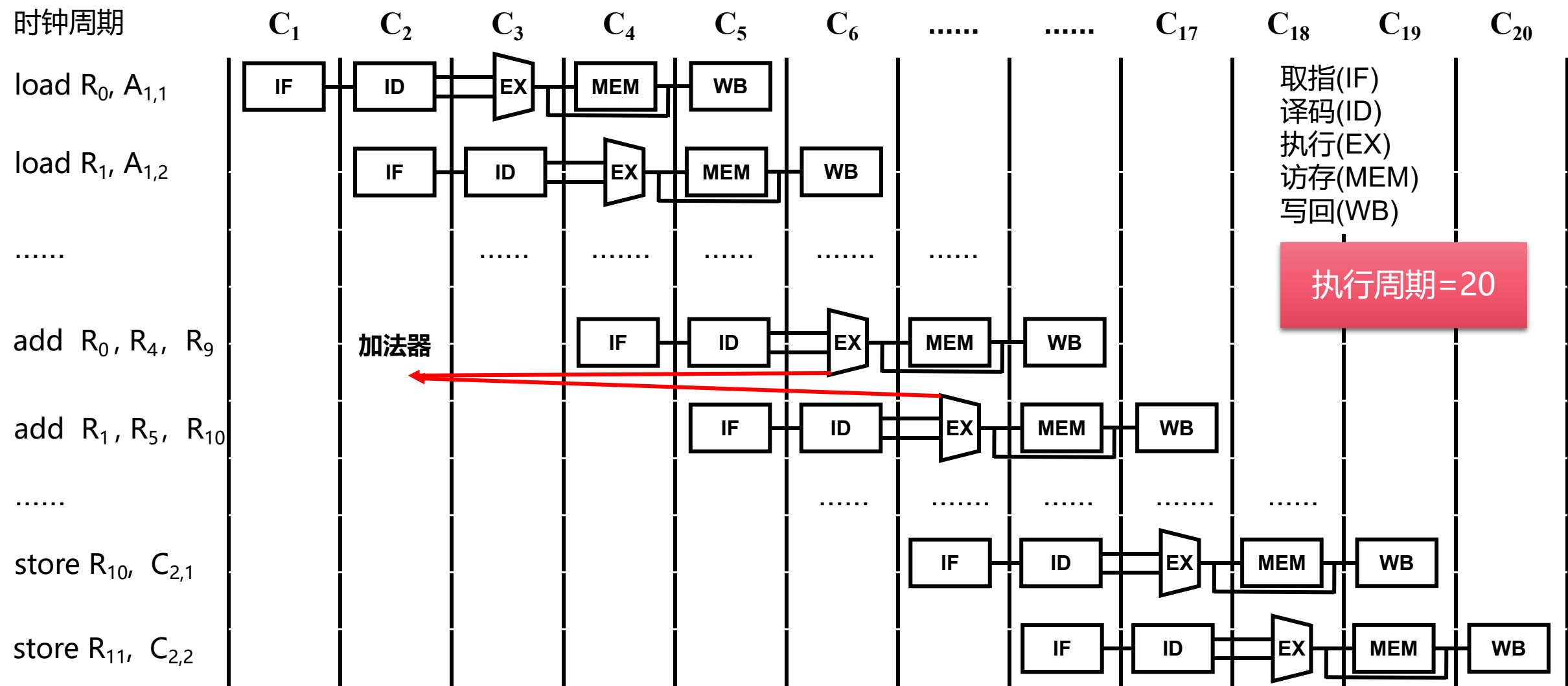
store R_8, C_{11}

store R_9, C_{12}

store R_{10}, C_{21}

store R_{11}, C_{22}

处理器实现



问题1：应用需求的发展变化

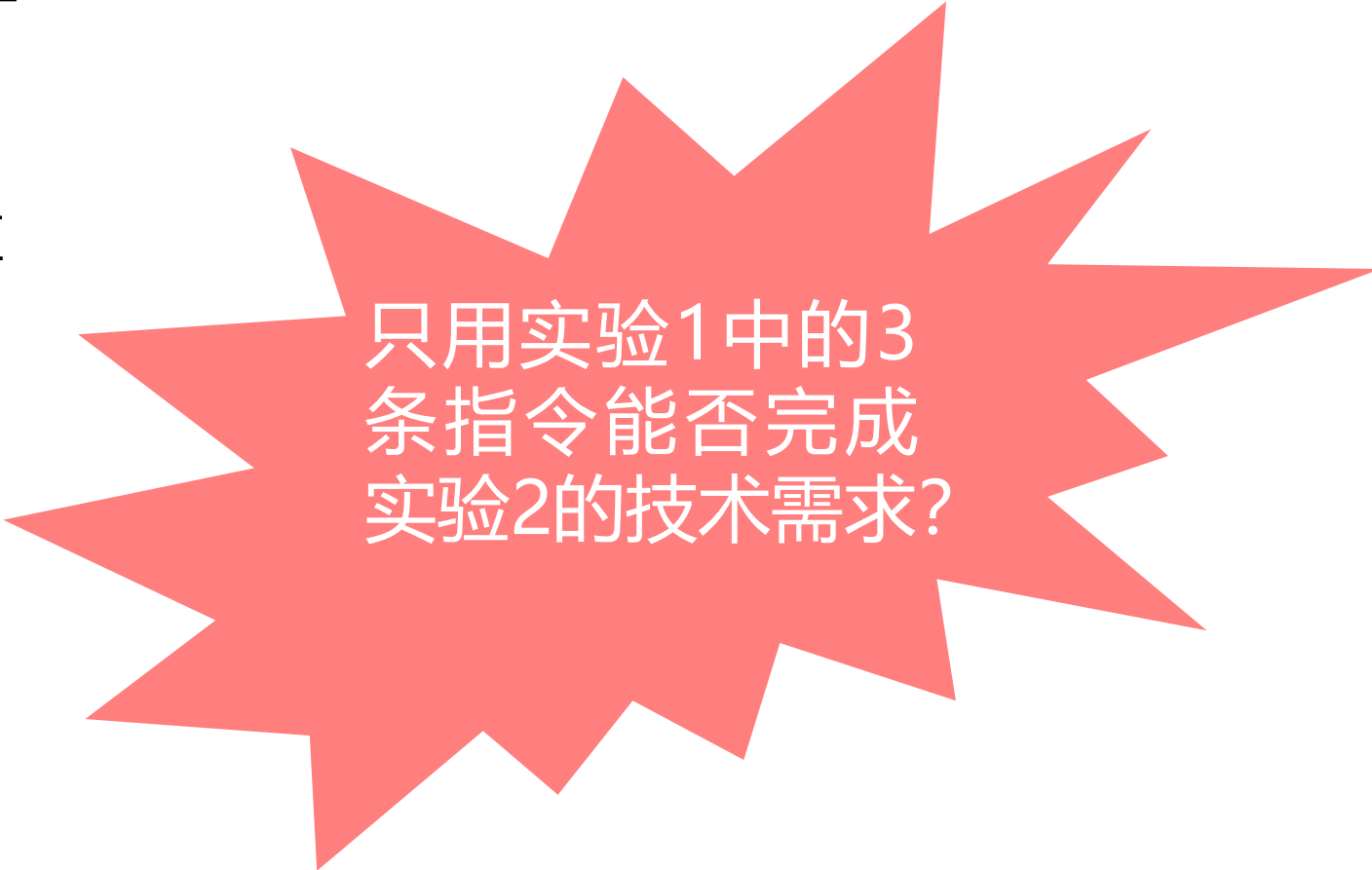
第 8 页

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

运算公式： $A \times B = C$

实例2的要求：

- 处理器一次只能完成1次算术运算
- 设计要求：
 - ◆ 指令集定义
 - ◆ 执行序列分析



只用实验1中的3
条指令能否完成
实验2的技术需求？

■四个指令

◆数字传输类指令

□ 输入数据 A

load R_A, A

□ 输出数据 B

store R_B, B

◆运算类指令

□ 两个数相加 $A+B=C$

add R_A, R_B, R_C

□ 两个数相乘 $A \times B = C$

mul R_A, R_B, R_C

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \quad A \times B = C$$

load $R_0, A_{1,1}$

load $R_1, A_{1,2}$

load $R_2, A_{2,1}$

load $R_3, A_{2,2}$

load $R_4, B_{1,1}$

load $R_5, B_{1,2}$

load $R_6, B_{2,1}$

load $R_7, B_{2,2}$

mul R_0, R_4, R_8

mul R_1, R_6, R_9

mul R_0, R_5, R_{10}

mul R_1, R_7, R_{11}

mul R_2, R_4, R_{12}

mul R_3, R_6, R_{13}

mul R_2, R_5, R_{14}

mul R_3, R_7, R_{15}

add R_8, R_9, R_0

add R_{10}, R_{11}, R_1

add R_{12}, R_{13}, R_2

add R_{14}, R_{15}, R_3

store R_0, C_{11}

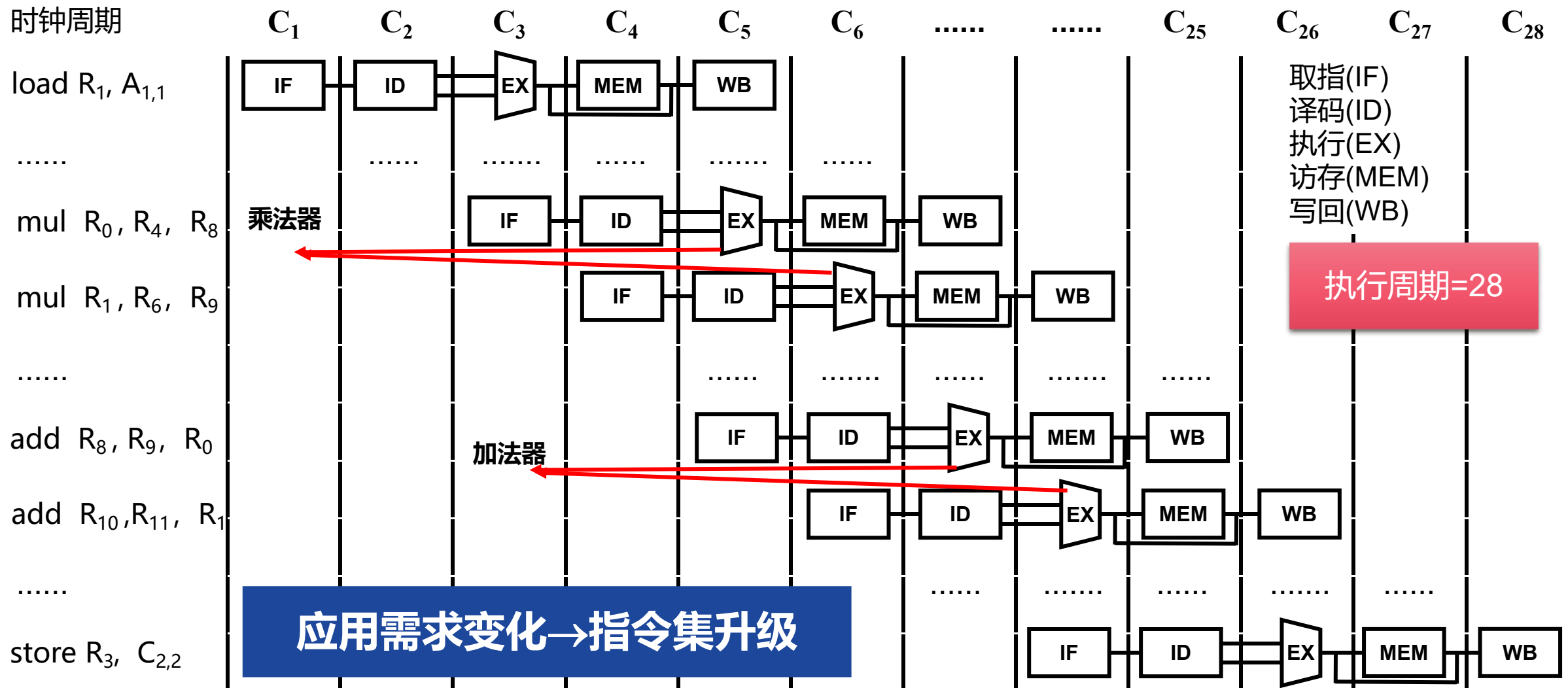
store R_1, C_{12}

store R_2, C_{21}

store R_3, C_{22}

处理器设计优化

第 10 页



问题2：指令系统设计的优化问题

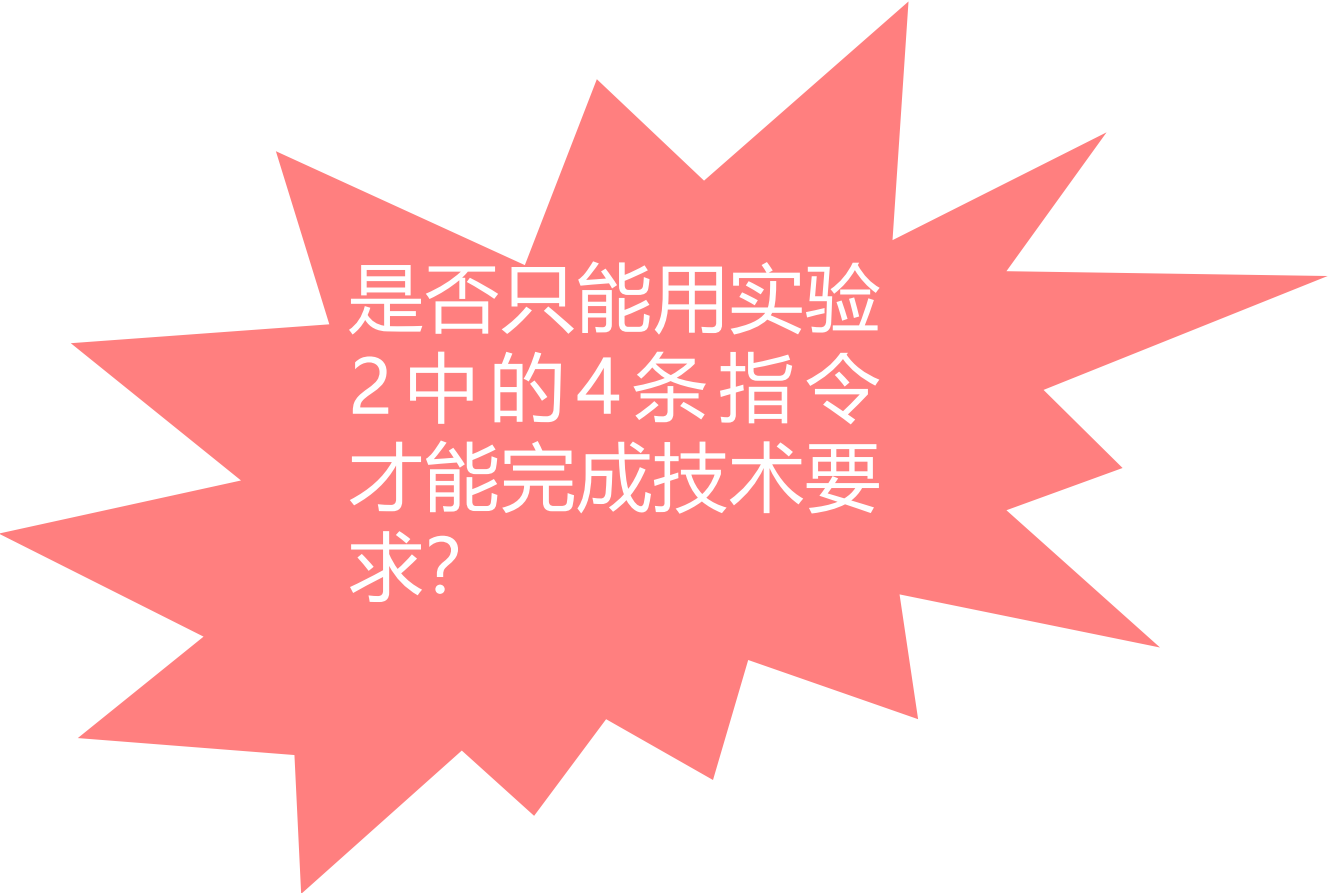
第 11 页

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

运算公式： $A \times B = C$

实例2的要求：

- 处理器一次只能完成1次算术运算
- 设计要求：
 - ◆ 指令集定义
 - ◆ 执行序列分析



是否只能用实验
2中的4条指令
才能完成技术要
求？

新型指令系统的设计

第 12 页

■四个操作

◆数字传输类指令

□ 输入数据 A

load R_A, A

□ 输出数据 B

store R_B, B

◆运算类指令

□ 两个数相加 $A+B=C$

add R_A, R_B, R_C

□ 三个数相乘加 $A \times B + C = D$

mac R_A, R_B, R_C, R_D

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$A \times B = C$$

load $R_0, A_{1,1}$

load $R_1, A_{1,2}$

load $R_2, A_{2,1}$

load $R_3, A_{2,2}$

load $R_4, B_{1,1}$

load $R_5, B_{1,2}$

load $R_6, B_{2,1}$

load $R_7, B_{2,2}$

mac R_0, R_4, R_{15}, R_8

mac R_1, R_6, R_8, R_{11}

mac R_0, R_5, R_{15}, R_8

mac R_1, R_7, R_8, R_{12}

mac R_2, R_4, R_{15}, R_8

mac R_3, R_6, R_8, R_{13}

mac R_2, R_5, R_{15}, R_8

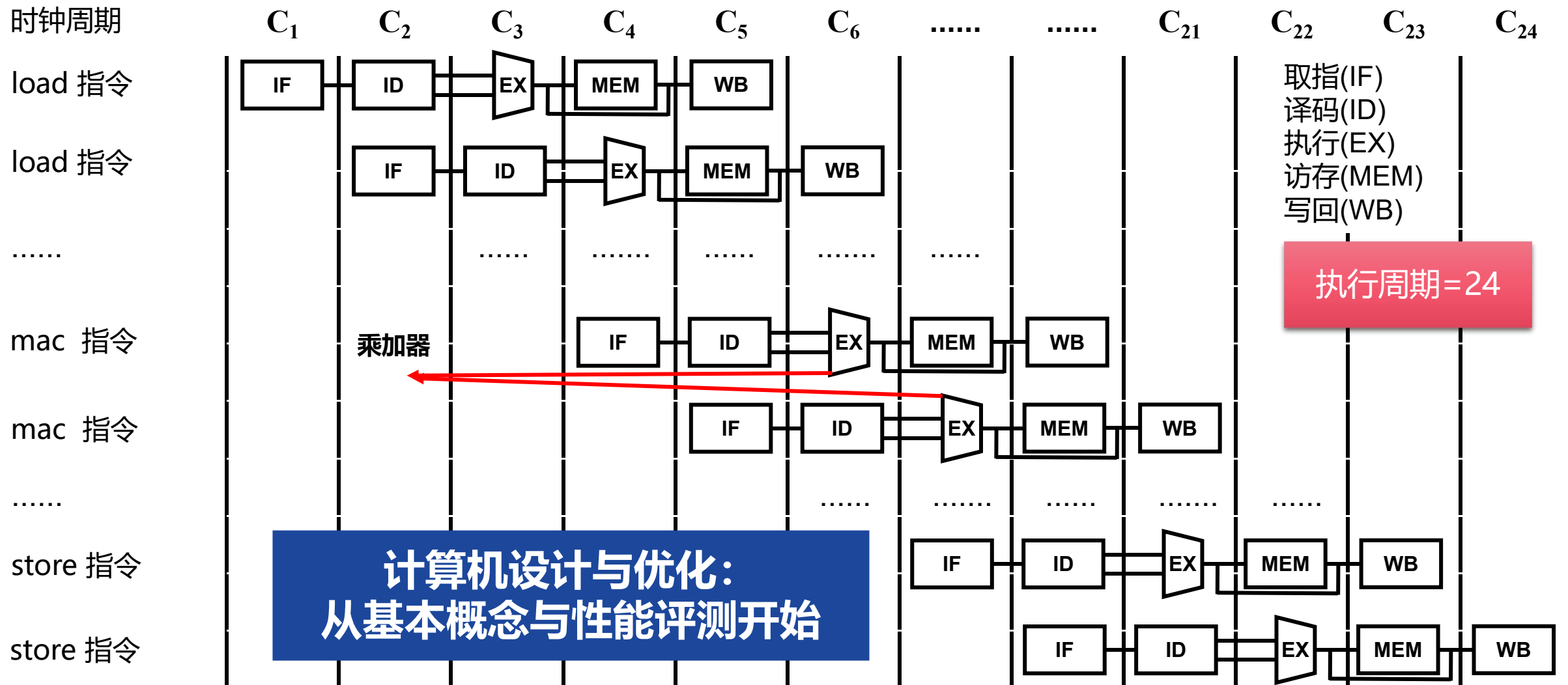
mac R_3, R_7, R_8, R_{14}

store R_{11}, C_{11}

store R_{12}, C_{12}

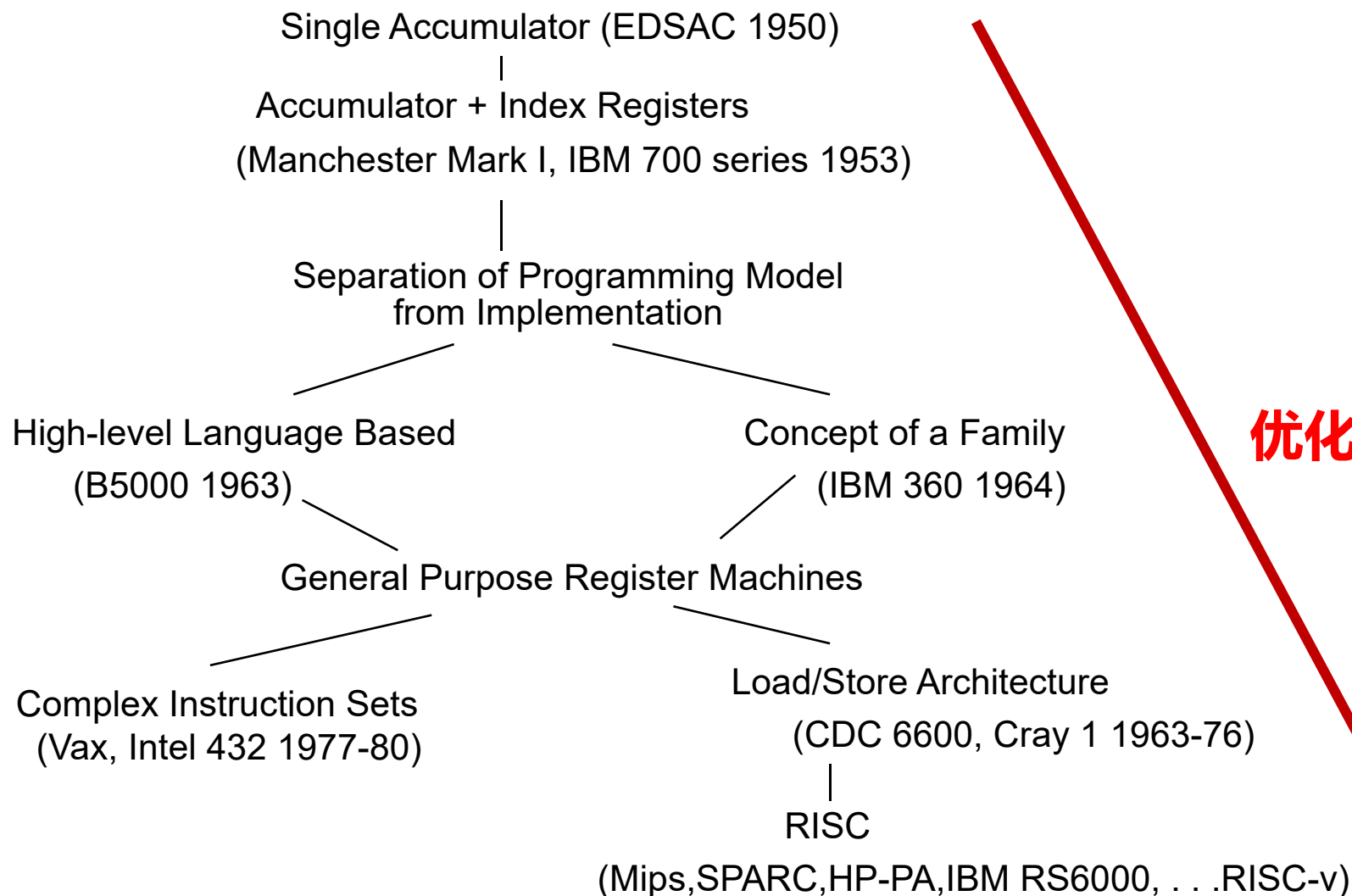
store R_{13}, C_{21}

store R_{14}, C_{22}



指令系统的演化

第 14 页



本章的重点是理解指令系统的功能设计与格式设计以及优化方法，并介绍一个经典指令系统。

2.1 [指令系统结构的分类](#)

2.2 [寻址方式](#)

2.3 [指令系统的设计与优化](#)

2.4 [指令系统的发展和改进](#)

2.5 [操作数类型和大小](#)

2.6 [典型指令系统 \(RISC-V\)](#)

2.1 指令系统结构的分类

2.1 指令系统结构的分类

第 17 页

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \quad \text{矩阵 } A \times B = C$$

程序=按一定顺序组成的指令序列

load R₀, A_{1,1}

load R₁, A_{1,2}

load R₂, A_{2,1}

load R₃, A_{2,2}

load R₄, B_{1,1}

load R₅, B_{1,2}

load R₆, B_{2,1}

load R₇, B_{2,2}

mul R₀, R₄, R₈

mul R₁, R₆, R₉

mul R₀, R₅, R₁₀

mul R₁, R₇, R₁₁

mul R₂, R₄, R₁₂

mul R₃, R₆, R₁₃

mul R₂, R₅, R₁₄

mul R₃, R₇, R₁₅

add R₈, R₉, R₀

add R₁₀, R₁₁, R₁

add R₁₂, R₁₃, R₂

add R₁₄, R₁₅, R₃

store R₀, C₁₁

store R₁, C₁₂

store R₂, C₂₁

store R₃, C₂₂

□方案一：两个数相乘 $A \times B = C$

mul R_A, R_B, R_C

28

□方案二：三个数相乘加 $A \times B + C = D$

mac R_A, R_B, R_C, R_D

24

区分指令系统的关键

1、存储机制

2、计算能力

2.1 指令系统结构的分类

第 18 页

1. 区别不同指令系统的主要因素：

- CPU用来存储操作数的存储单元类型

2. CPU中用来存储操作数的存储单元：

- 堆栈、累加器、通用寄存器组

3. 将指令系统的结构分为三种类型：

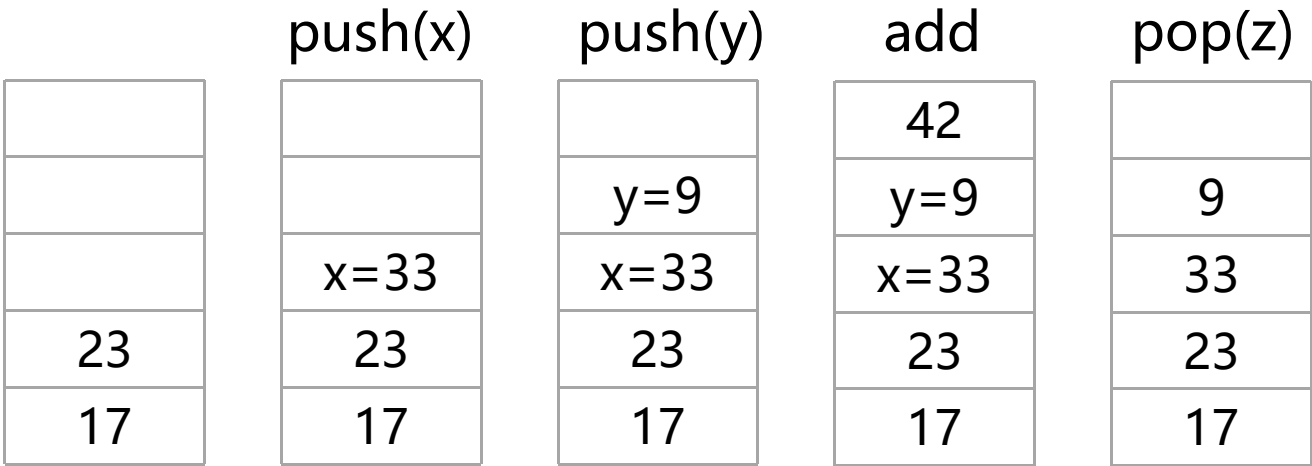
- 堆栈结构、累加器结构、通用寄存器结构
- 对于不同结构，操作数的位置、个数以及操作数的给出方式不同
 - ✓ 显式给出：用指令字中的操作数字段给出
 - ✓ 隐式给出：使用事先约定好的单元

关注要点

2.1 指令系统结构的分类

例1: 表达式 $Z=X+Y$ 在4种类型指令系统结构上的代码。假设: X、Y、Z均保存在存储器单元中, 并且不能破坏X和Y的值。

① 堆栈是一个特定的存储电路, 一端是固定的, 另一端是浮动的 数据存入或取出, 只能在浮动的一端 (称栈顶) 进行, 按照 “先进后出” 的原则存取, 位于其中间的元素, 必须在其栈上部 (后进栈者) 诸元素逐个移出后才能取出。



堆栈结构	优点	一种表示计算的简单模型; 指令短小; 隐式给出
	缺点	不能随机访问堆栈, 从而很难生成有效代码; 堆栈难以高效实现

2.1 指令系统结构的分类

第 20 页

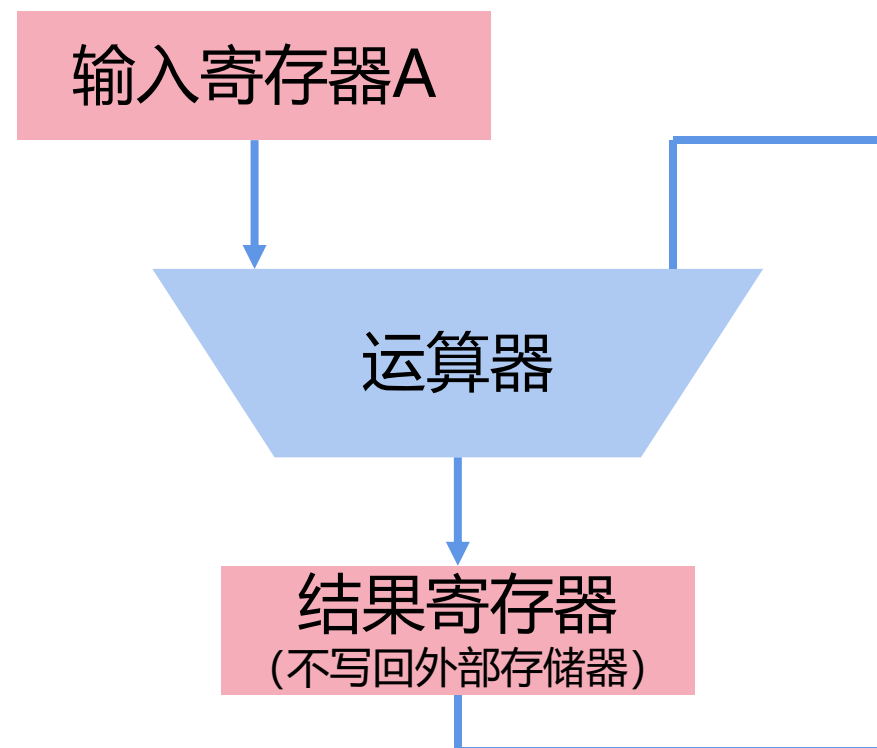
- ② 累加器 (accumulator) 是一种寄存器，专门存放算术或逻辑运算的一个操作数和运算结果的寄存器。

加、减、读出、移位、
循环移位和求补等操作

```
load x  
add y  
store z
```

$$z = x + y$$

x



累加器结构	优点	减少了机器的内部状态；指令短小；隐式给出
	缺点	累加器是唯一的暂存器，存储器通信开销最大

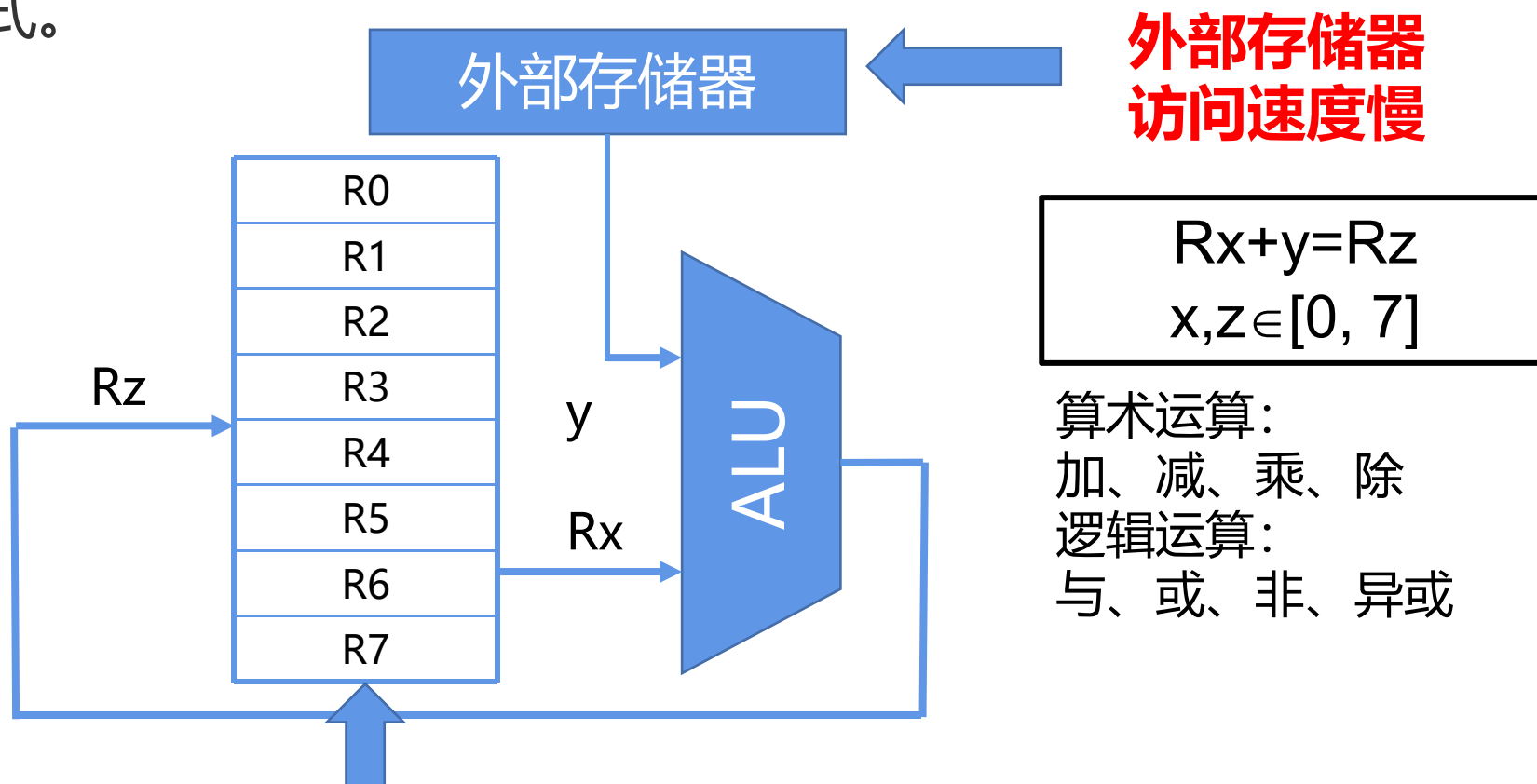
2.1 指令系统结构的分类

第 21 页

- ③ 通用寄存器 (register) 可用于传送和暂存数据, 也可参与算术逻辑运算, 并保存运算结果, 有两种形式。

形式一: 寄存器-存储器结构

```
load  R1, X
add   R1, Y
store R1, Z
```



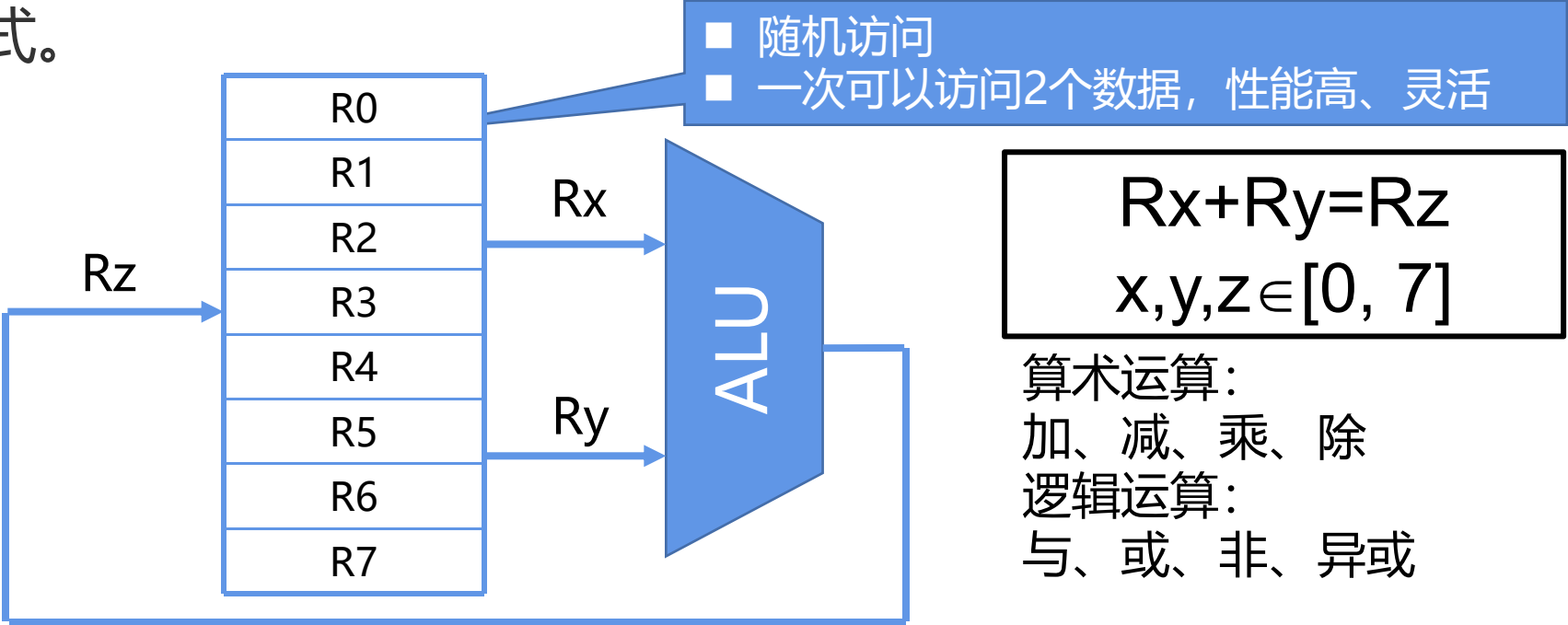
相对于累加器结构, 增加了内部寄存器数量, 降低了存储器通信开销

2.1 指令系统结构的分类

③ 通用寄存器 (register) 可用于传送和暂存数据，也可参与算术逻辑运算，并保存运算结果，有两种形式。

形式二：寄存器-寄存器结构

```
load R1, X
load R2, Y
add R3, R1, R2
store R3, Z
```



通用寄存器结构	优点	①寄存器的访问速度比存储器快，减少对存储器的访问，加快程序的执行速度； ②用更少的地址位对寄存器进行寻址，有效地减少程序目标代码的大小； ③编译器能更有效地分配和使用寄存器，易于生成高效的目标代码
	缺点	所有操作数均需命名，要显式表示，指令比较长

2.1 指令系统结构的分类

第 23 页

5. 根据ALU指令的操作数的两个特征对通用寄存器型结构进一步细分

■ ALU指令的操作数个数

- 3个操作数的指令：两个源操作数、一个目的操作数
- 2个操作数的指令：其中一个操作数既作为源操作数，又作为目的操作数。

ALU指令中存储器操作数的个数	ALU指令中操作数的最多个数	结构类型	机器实例
0	3	RR	MIPS, SPARC, Alpha, PowerPC, AR
1	2	RM	IBM 360/370, Intel 80x86, Motorola 68000
	3	RM	IBM 360/370

6. 寄存器-寄存器型结构（RR结构）

- 现代指令系统结构的主流，在灵活性和提高性能方面有明显的优势
- 称为load-store结构，即只有load指令和store指令能够访问存储器。（所有操作数都是来自通用寄存器组）
 - **优点：** 指令字长固定，指令结构简洁，是一种简单的代码生成模型，因此各种指令的执行时钟周期数相近；
 - **缺点：** 与指令中含存储器操作数的指令系统结构相比，指令条数多，目标代码不够紧凑，因而程序占用的空间比较大。

2.2 寻址方式

2.2 寻址方式

第 26 页

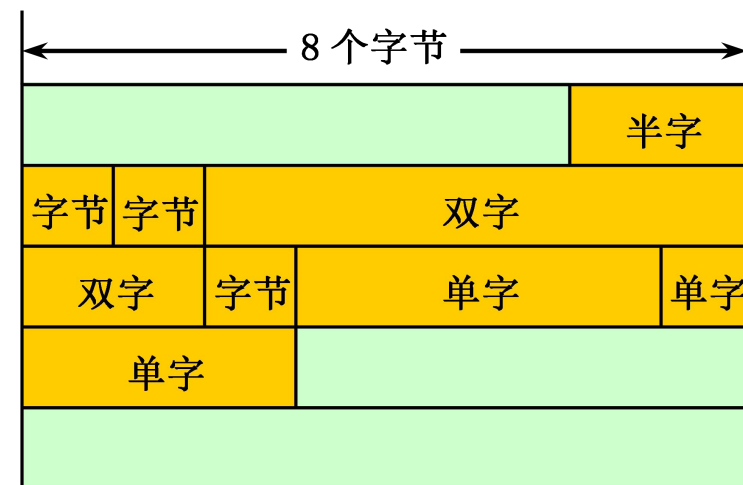
1. 一个需要注意的问题：①物理地址空间的信息如何存放？如何在存储器中存放不同宽度的信息？

以IBM370为例子进行讨论。

- 信息有字节、半字（双字节）、单字（4字节）和双字（8字节）等宽度。
- 主存宽度为8个字节：采用按字节编址，各类信息都是用该信息的首字节地址来寻址。

方式一：允许它们任意存储

- 很可能会出现一个信息跨存储字边界而存储于两个存储单元中



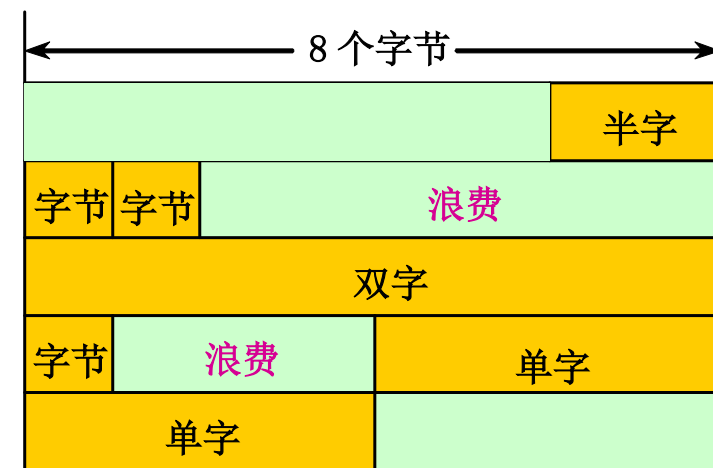
(a)

2.2 寻址方式

第 27 页

方式二：不允许它们任意存储，即信息宽度不超过主存宽度的信息，必须存放在一个存储字内，不能跨边界。

信息存储的整数边界概念：信息在主存中存放的起始地址必须是该信息宽度（字节数）的整数倍



(b)

■满足以下条件：

➤字节信息的起始地址为： $x \dots x x x x$

半字信息的起始地址为： $x \dots x x x 0$

➤单字信息的起始地址为： $x \dots x x 0 0$

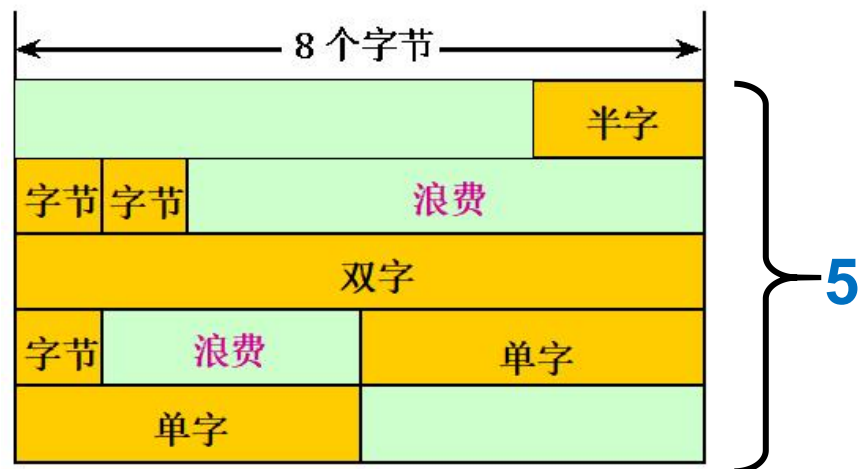
双字信息的起始地址为： $x \dots x 0 0 0$

■优缺点：存在存储空间的浪费，但保证访问速度。

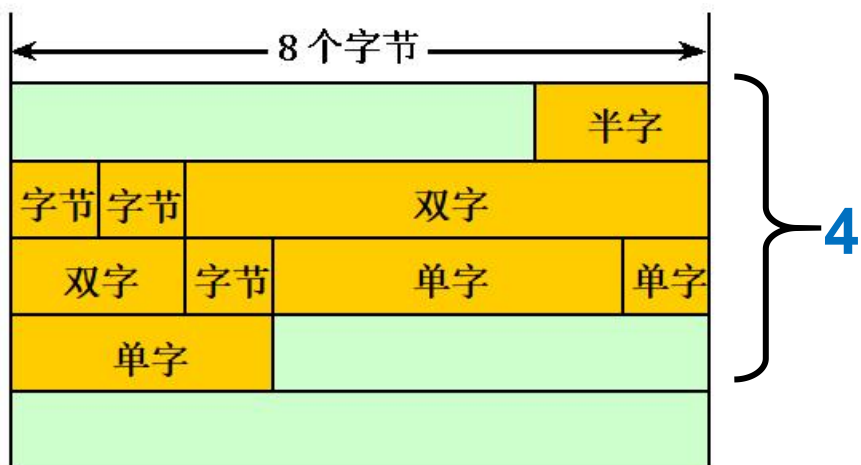
2.2 寻址方式

第 28 页

存储效率指计算机中数据本身所需的存储量和整个数据所占的存储量之比。



$$\begin{aligned}\text{存储效率} &= \frac{\text{整个数据所需的存储量}}{\text{整个数据所占的存储量}} \\ &= \frac{2+1+1+8+1+4+4}{8 \times 5} = \frac{21}{40} = 52.5\%\end{aligned}$$



$$\text{存储效率} = \frac{2+1+1+8+1+4+4}{8 \times 4} = \frac{21}{32} = 65.625\%$$

问题：通常以牺牲性能为代价来增加效率
方法：折中

2.2 寻址方式

第 29 页

指令格式：用二进制代码表示指令的结构形式。

格式一：



操作码表示操作的性质和功能，操作数码表示指令操作的对象

格式二：



地址码的位数表示了直接访问的最大存储空间。

1. 寻址方式：指令系统中如何形成所要访问的数据的地址。

- 寻址方式可指明指令中的操作数是一个常数、一个寄存器操作数或者是一个存储器操作数。
- 对于存储器操作数来说，由寻址方式确定的存储器地址称为有效地址。

2. 寻址方式的方法

- 将寻址方式编码于操作码中，由操作码描述相应操作的寻址方式，适合处理机采用load-store结构
- 在指令字中设置专门的寻址字段，用以直接指出寻址方式。
 - 灵活，操作码短，但需要设置专门的寻址方式字段，而且操作码和寻址方式字段合起来所需要的总位数可能会比隐含方法的总位数多。
 - 适合：处理机具有多种寻址方式，且指令有多个操作数。

MIPS处理器I型指令（立即数寻址）

汇编语言： `addi s1,s2, 10`

表示： $\$s1 = \$s2 + E(10)$

MIPS处理器R型指令（寄存器寻址）

汇编语言： `add $t0, $s1, s 2`

表示： $t0 = \$s1 + \$s2$

2.2 寻址方式

第 31 页

七种基本寻址方式：

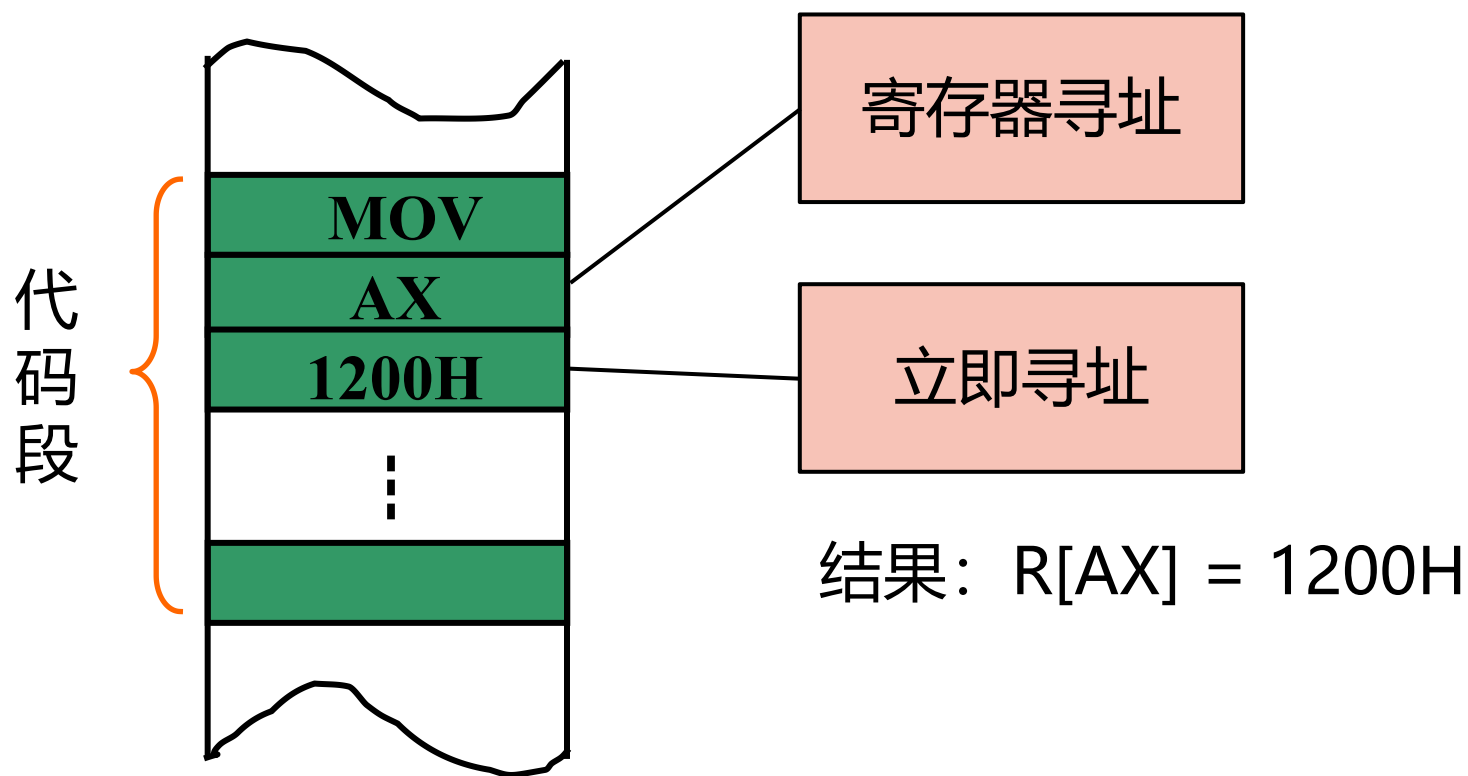
立即数寻址：	立即数 → 寄存器
直接寻址：	[偏移地址] → 寄存器
寄存器寻址：	寄存器 → 寄存器
寄存器间接寻址：	[基址寄存器/变址寄存器] → 寄存器
寄存器相对寻址：	[基址寄存器/变址寄存器+偏移量值] → 寄存器
基址变址寻址：	[基址寄存器+变址寄存器] → 寄存器
相对基址变址寻址：	[基址寄存器+变址寄存器+偏移量值] → 寄存器

2.2 寻址方式

第 32 页

立即数寻址：指令中的源操作数是立即数，即源操作数是参加操作的数据本身

例：MOV AX, 1200H



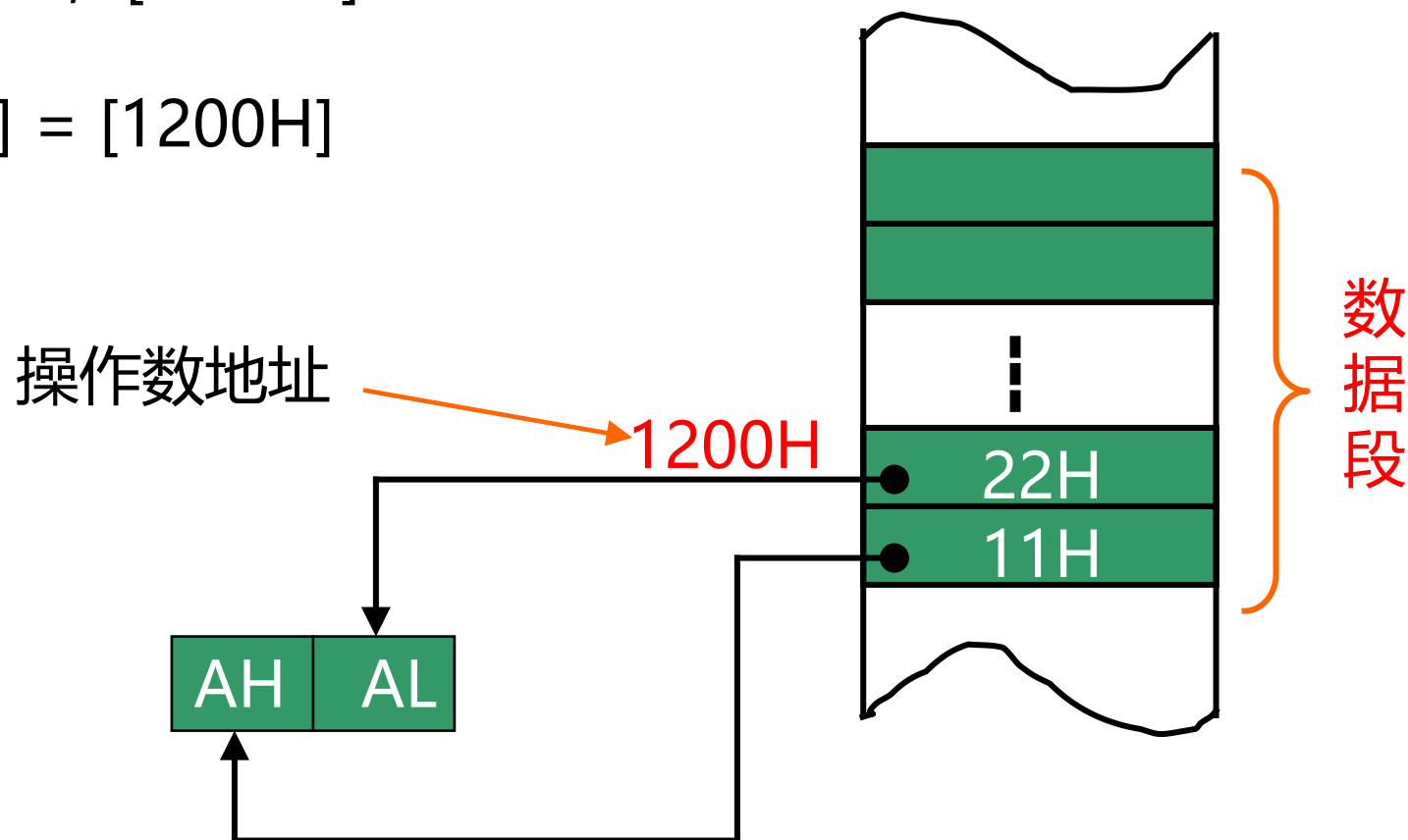
2.2 寻址方式

第 33 页

直接寻址：指令中的源操作数段中直接给出操作数的地址

例：MOV AX, [1200H]

结果：R[AX] = [1200H]



2.2 寻址方式

第 34 页

寄存器寻址方式：指令所需的操作数存储在某寄存器中，或把目标操作数存入寄存器，把在指令中指出所使用寄存器（即寄存器的助忆符）的寻址方式。

例如：指令集可以引用的寄存器及其符号名称如下：

16位寄存器：AX、BX、CX、DX、SI、DI、SP、DP

32位寄存器：EAX、EBX、ECX、EDX、ESI、EDI、ESP、EDP

1、源操作数是寄存器寻址方式： ADD VARW EAX MOV VARH, BX

其中VARW和VARH分别代表字和半字的内存变量

2、目标操作数是寄存器寻址方式： ADD AX 1234h MOV EBX 12345678h

3、源和目标操作数是寄存器寻址方式： MOV AX BX MOV EAX EBX

2.2 寻址方式

第 35 页

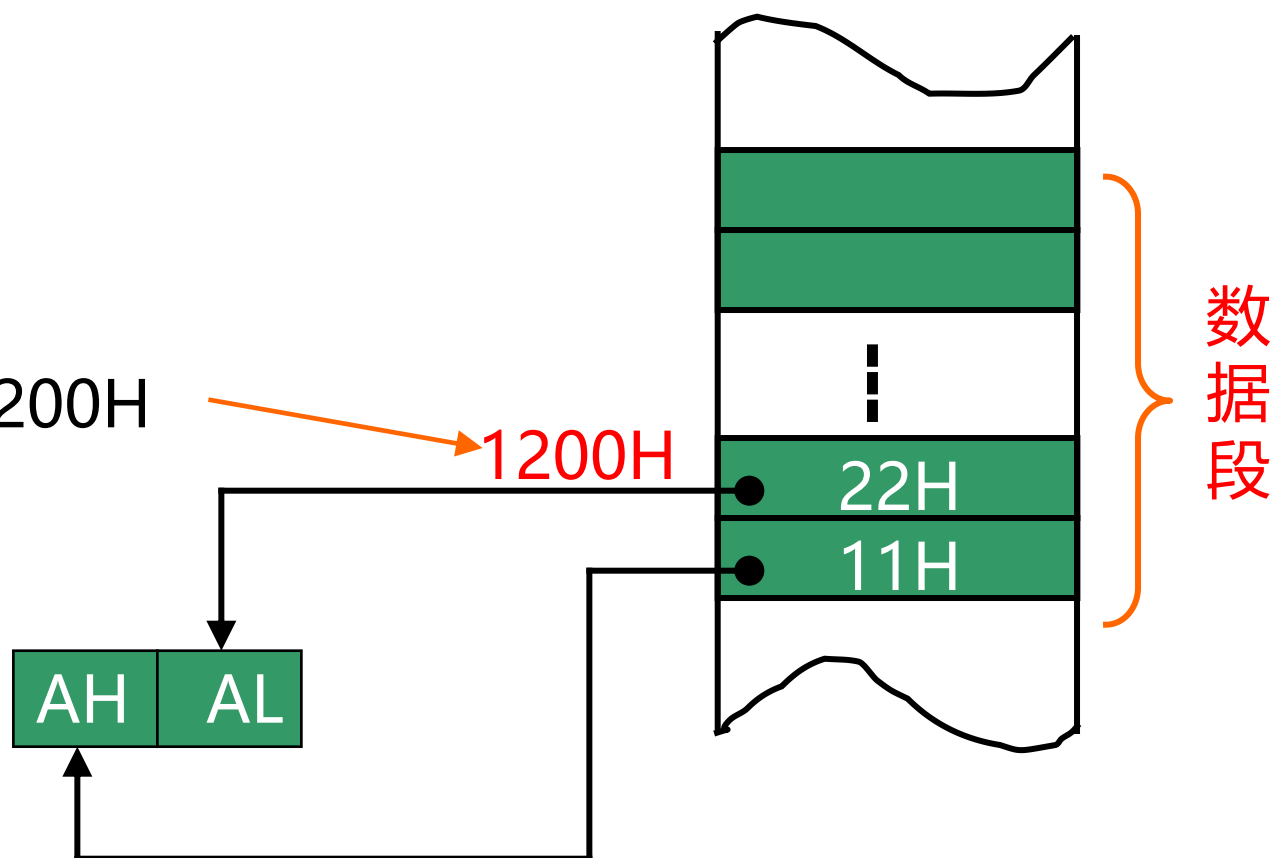
寄存器间接寻址：指令中参与操作的操作数存放在内存中，其地址为指令中的基址寄存器BX的内容

例：MOV AX, [BX]

设BX=1200H

结果：R[AX] = [1200H]

[BX]=1200H



2.2 寻址方式

第 36 页

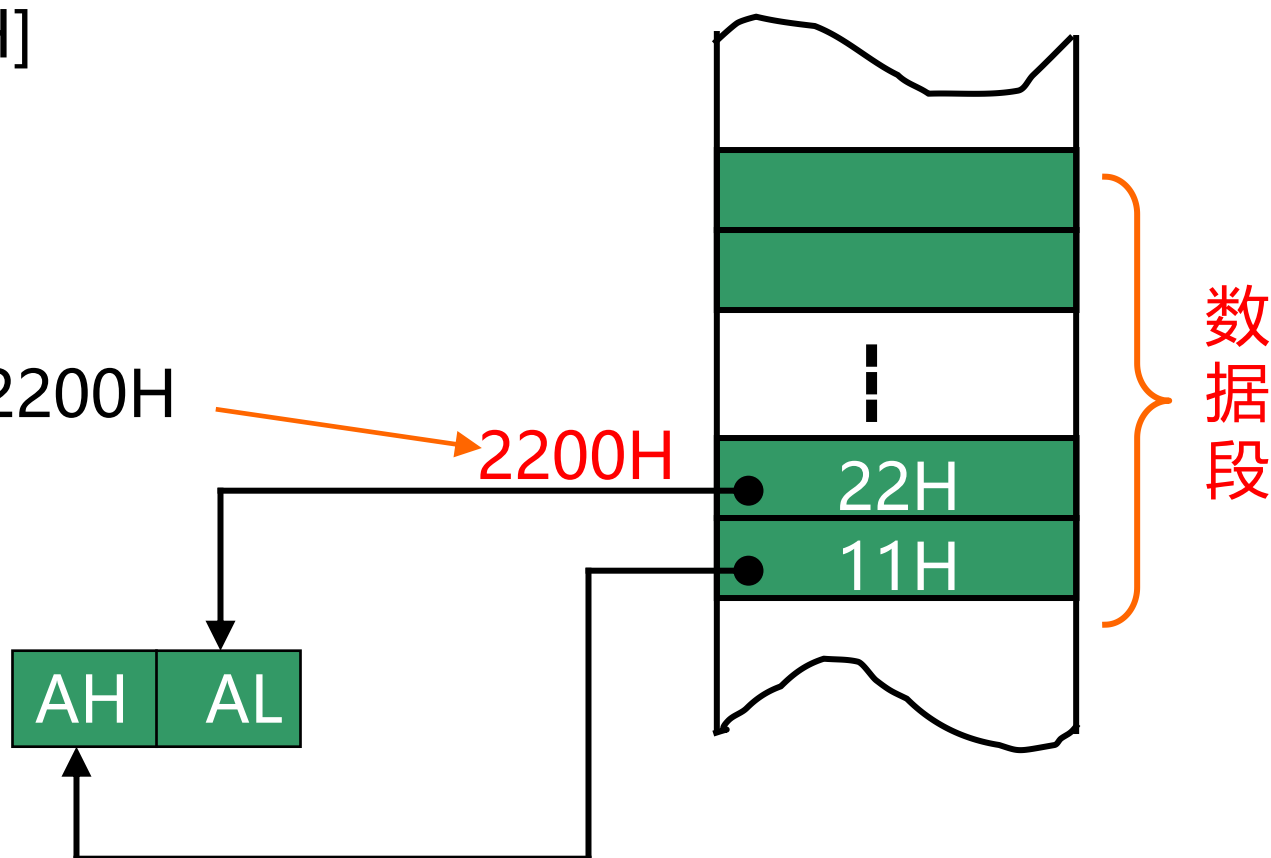
寄存器相对寻址：指令中参与操作的操作数存放在内存中，其地址为指令中的基址寄存器BX和偏移量DI的内容

例：MOV AX, [BX+1000H]

设BX=1200H

结果：R[AX] = [2200H]

[BX+1000H]=2200H



2.2 寻址方式

第 37 页

基址变址寻址：指令中参与操作的操作数存放在内存中，其地址为指令中的基址寄存器BX和变址寄存器SI的内容

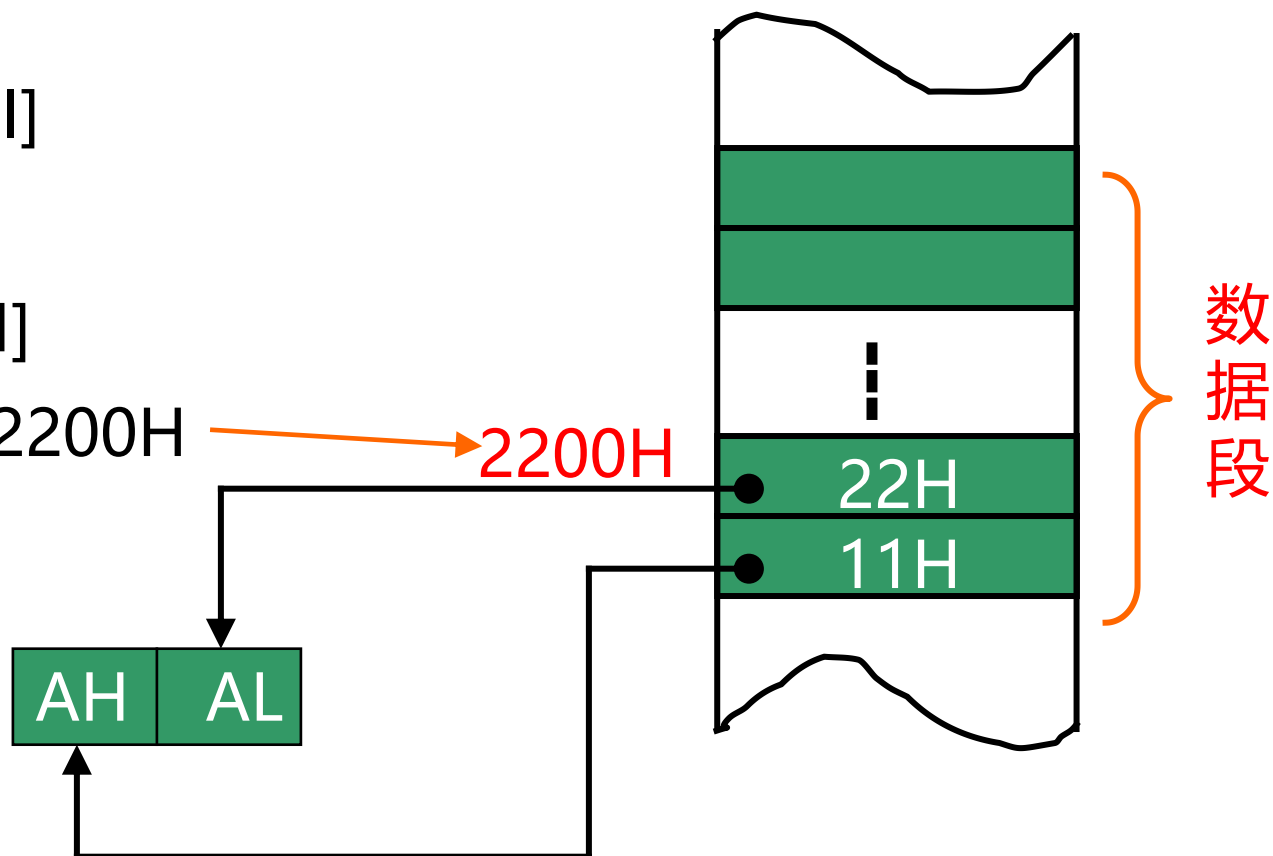
例：MOV SI, 1000H

MOV AX, [BX+SI]

设BX=1200H

结果：R[AX] = [2200H]

[BX+SI]=2200H



2.2 寻址方式

第 38 页

相对基址变址寻址：指令中参与操作的操作数存放在内存中，其地址为指令中基址寄存器BX、变址寄存器SI和偏移量DI的内容

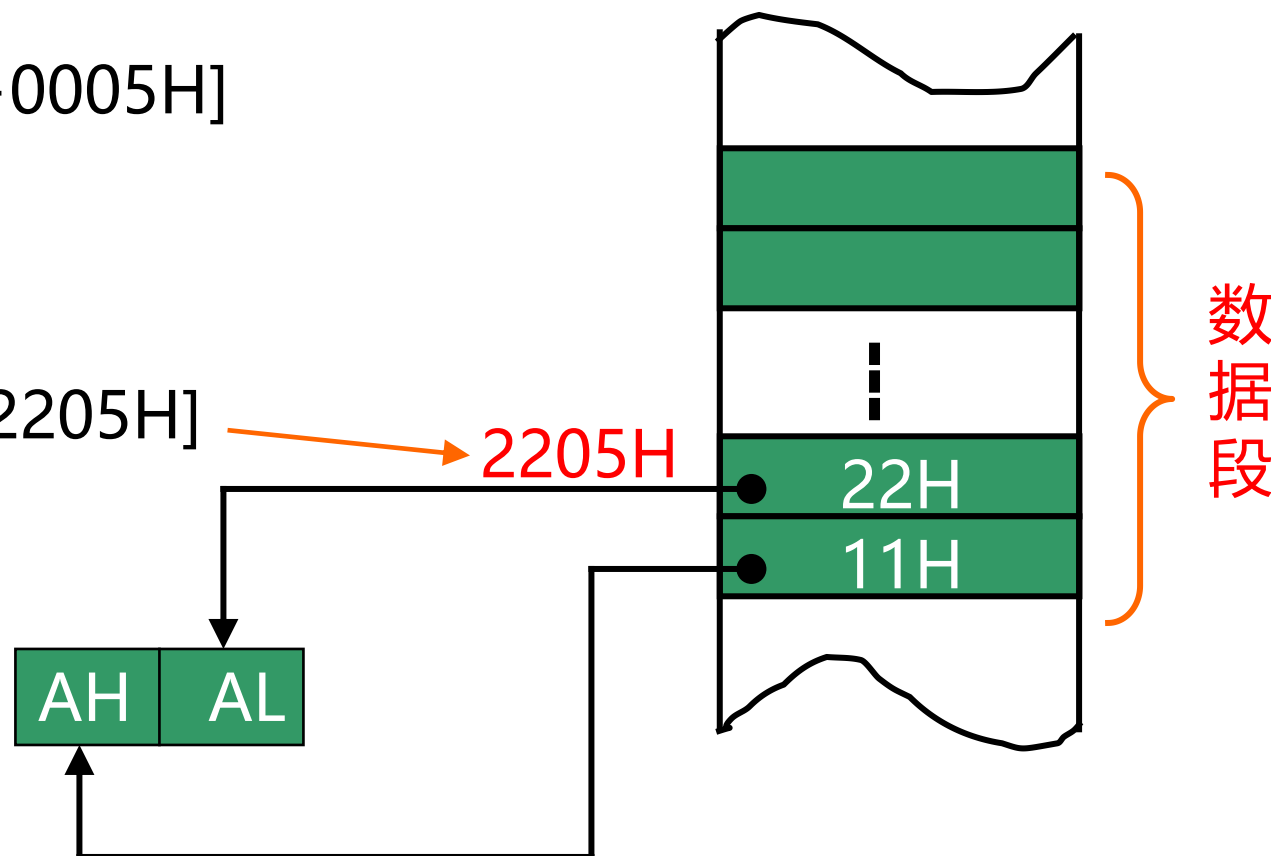
例：MOV SI, 1000H

MOV AX, [BX+SI+0005H]

设BX=1200H

[BX+SI]=2205H

结果：R[AX] = [2205H]



2. 设计实例：VAX机器的操作数寻址方式

- $\leftarrow$ ：赋值操作
- Mem：存储器
- Regs：寄存器组
- 方括号：表示内容
 - Mem[]：存储器的内容
 - Regs[]：寄存器的内容
 - Mem[Regs[R1]]：以寄存器R1中的内容作为地址的存储器单元中的内容

2.2 寻址方式

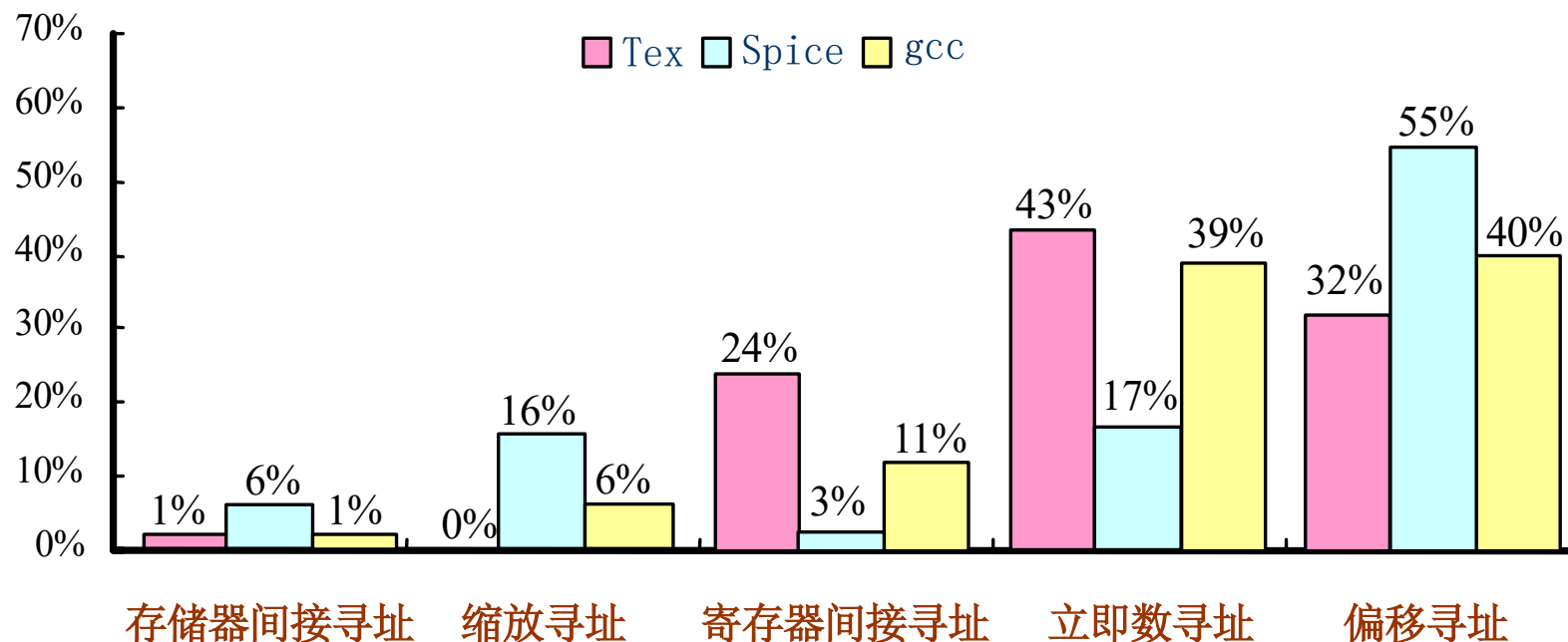
第 40 页

寻址方式	指令实例	含义
立即值寻址	ADD R3 , #6	$\text{Regs}[\text{R3}] \leftarrow \text{Regs}[\text{R3}] + 6$
直接寻址或绝对寻址	ADD R4 , (1010)	$\text{Regs}[\text{R4}] \leftarrow \text{Regs}[\text{R4}] + \text{Mem}[1010]$
寄存器寻址	ADD R1 , R2	$\text{Regs}[\text{R1}] \leftarrow \text{Regs}[\text{R1}] + \text{Regs}[\text{R2}]$
寄存器间接寻址	ADD R4 , (R2)	$\text{Regs}[\text{R4}] \leftarrow \text{Regs}[\text{R4}] + \text{Mem}[\text{Regs}[\text{R2}]]$
偏移寻址	ADD R3 , 120(R2)	$\text{Regs}[\text{R3}] \leftarrow \text{Regs}[\text{R3}] + \text{Mem}[120 + \text{Regs}[\text{R2}]]$
索引寻址	ADD R4 , (R2 + R3)	$\text{Regs}[\text{R4}] \leftarrow \text{Regs}[\text{R4}] + \text{Mem}[\text{Regs}[\text{R2}] + \text{Regs}[\text{R3}]]$
存储器间接寻址	ADD R2 , @(R4)	$\text{Regs}[\text{R2}] \leftarrow \text{Regs}[\text{R2}] + \text{Mem}[\text{Mem}[\text{Regs}[\text{R4}]]]$
自增寻址	ADD R1 , (R2)+	$\begin{aligned} \text{Regs}[\text{R1}] &\leftarrow \text{Regs}[\text{R1}] + \text{Mem}[\text{Regs}[\text{R2}]] \\ \text{Regs}[\text{R2}] &\leftarrow \text{Regs}[\text{R2}] + d \end{aligned}$
自减寻址	ADD R1 , -(R2)	$\begin{aligned} \text{Regs}[\text{R2}] &\leftarrow \text{Regs}[\text{R2}] - d \\ \text{Regs}[\text{R1}] &\leftarrow \text{Regs}[\text{R1}] + \text{Mem}[\text{Regs}[\text{R2}]] \end{aligned}$
缩放寻址	ADD R1 , 80(R2)[R3]	$\text{Regs}[\text{R1}] \leftarrow \text{Regs}[\text{R1}] + \text{Mem}[80 + \text{Regs}[\text{R2}] + \text{Regs}[\text{R3}] * d]$

2.2 寻址方式

第 41 页

3. 各种寻址方式的使用情况统计结果



立即数寻址方式和偏移寻址方式的使用频度最高。

在VAX机器上运行gcc、Spice和Tex 基准程序

采用多种寻址方式可以显著地减少程序的指令条数，但可能增加计算机的实现复杂度以及指令的CPI——折中方案。

4. 立即数寻址方式

■ 立即数寻址方式的使用频度

指令类型	使用频度	
	整型平均	浮点平均
load指令	23%	22%
ALU指令	25%	19%
所有指令	21%	16%

大约1/4的load指令和ALU指令采用了立即数寻址。

结论：基本寻址方式占比大——①优化方向②方法折中

2.3 指令系统的设计与优化

2.3.1 指令系统设计的基本原则

第 44 页

1. 指令系统的设计

- 首先考虑所应实现的基本功能，确定哪些基本功能应该由硬件实现，哪些功能由软件实现比较合适，具体包括：

- ✓ 指令的功能设计
- ✓ 指令格式的设计

□方案二：三个数相乘加 $A \times B + C = D$
mac R_A, R_B, R_C, R_D

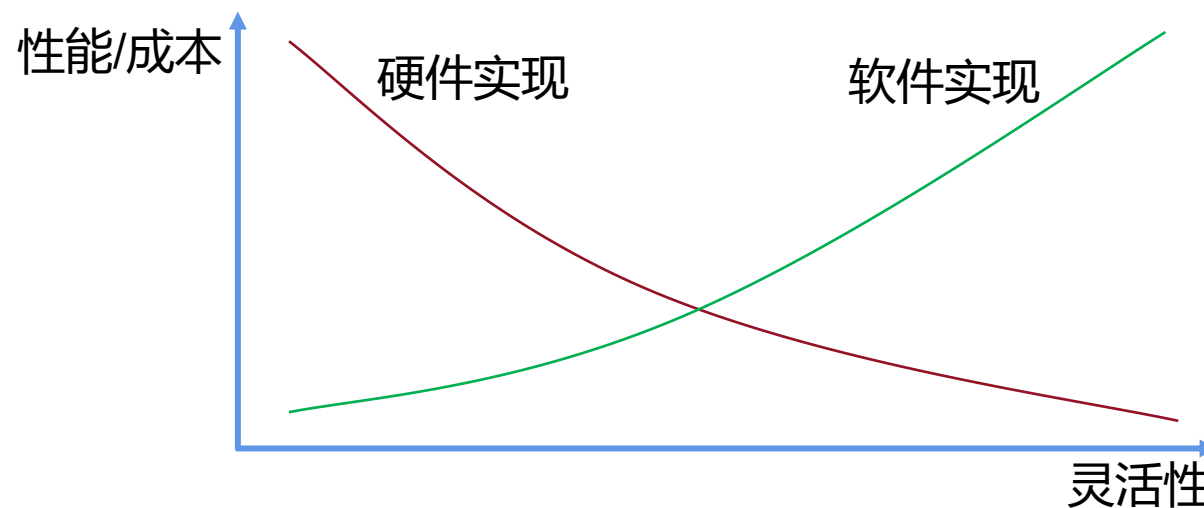
2. 在确定哪些基本功能用硬件来实现时，主要考虑3个因素：速度、成本、灵活性。

- 硬件实现的特点：

- ✓ 速度快、成本高、灵活性差

- 软件实现的特点：

- ✓ 速度慢、价格便宜、灵活性好



2.3.1 指令系统设计的基本原则

第 45 页

3. 对指令系统的基本要求：**完整性、规整性、正交性、高效率、兼容性**

- 完整性：在一个有限可用的存储空间内，对于任何可解的问题，编制计算程序时，指令系统所提供的指令足够使用，要求指令系统功能齐全、使用方便。

■ 三个指令

◆ 数字传输类指令

- 输入数据 A

load R_A, A

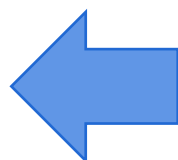
- 输出数据 B

store R_B, B

◆ 运算类指令

- 两个数相加 $A+B=C$

add R_A, R_B, R_C



$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \quad A+B=C$$

2.3.1 指令系统设计的基本原则

3. 对指令系统的基本要求：**完整性、规整性、正交性、高效率、兼容性**

- 完整性：在一个有限可用的存储空间内，对于任何可解的问题，编制计算程序时，指令系统所提供的指令足够使用，要求指令系统功能齐全、使用方便。
- 下表为许多指令系统结构都包含的一些指令类型，前4类属于通用计算机系统的基本指令，对于最后4种类型的操作，不同指令系统结构的支持大不相同。

操作类型	实例	操作类型	实例
算术和逻辑运算	算术/逻辑操作：加，减，乘，除，与，或等	浮点	浮点操作：加，减，乘，除，比较等
数据传输	load, store	十进制	十进制加/乘，十进制到字符的转换等
控制	分支，跳转，过程调用和返回，自陷等	字符串	字符串移动，字符串比较，字符串搜索等
系统	操作系统调用，虚拟存储器管理等	图形	像素操作，压缩/解压操作等

2.3.1 指令系统设计的基本原则

第 47 页

➤ 控制指令是用来改变控制流的。

- ✓ 跳转：当指令是无条件改变控制流时，称之为跳转指令。
- ✓ 分支：当控制指令是有条件改变控制流时，则称之为分支指令。

➤ 能够改变控制流的指令

- ✓ 分支
- ✓ 跳转
- ✓ 过程调用
- ✓ 过程返回

指令类型	使用频度	
	整型平均	浮点平均
调用/返回	19%	8%
跳转	6%	10%
分支	75%	82%

➤ 控制指令的使用频度

改变控制流的大部分指令是分支指令（条件转移）

- ✓ load-store型指令系统结构的机器，基准程序为SPEC CPU2000

2.3.1 指令系统设计的基本原则

第 48 页

➤ 分支条件的方法及其优缺点

名 称	检测分支条件的方法	优 点	缺 点
条件码 (CC)	检测由ALU操作设置的一些特殊的位 (即CC)	可以自由设置分支条件	条件码是增设的状态。而且它限制了指令的执行顺序, 因为要保证条件码能顺利地传送给分支指令。
条件寄存器	比较指令把比较结果放入任何一个寄存器, 检测时就检测该寄存器。	简单	占用了一个寄存器
比较与分支	比较操作是分支指令的一部分, 通常这种比较是受到一定限制的。	用 一 条 指 令 (而不是两条) 就能实现分支	当采用流水方式时, 该指令的操作可能太多, 在一拍内做不完。

2.3.1 指令系统设计的基本原则

第 49 页

➤ 转移目标地址的表示

- ✓ 最常用的方法：在指令中提供一个偏移量，由该偏移量和程序计数器（PC）的值相加而得出目标地址（PC相对寻址）
- ✓ 优点：有效地减少表示该目标地址所需要的位数；位置无关（代码可被装载到主存的任意位置执行）
- ✓ 关键：确定偏移量字段的长度。模拟结果表明：采用4～8位的偏移量字段（以指令字为单位）就能表示大多数控制指令的转移目标地址了。

2.3.1 指令系统设计的基本原则

第 50 页

➤ 过程调用和返回

- ✓ 除了要改变控制流之外，可能还要保存机器状态。至少也得保存返回地址（放在专用的链接寄存器或堆栈中）。
- ✓ 过去有些指令系统结构提供了专门的保存机制来保存许多寄存器的内容。
- ✓ 现在较新的指令系统结构则要求由编译器生成load和store指令来保存或恢复寄存器的内容。

2.3.1 指令系统设计的基本原则

第 51 页

■ 规整性：主要包括对称性和均匀性。

1. 对称性：所有与指令系统有关的存储单元的使用、操作码的设置等都是对称的。

例如：在存储单元的使用上，所有通用寄存器都要同等对待。在操作码的设置上，如果设置了A - B的指令，就应该也设置B - A的指令。

2. 均匀性：指对于各种不同的操作数类型、字长、操作种类和数据存储单元，指令的设置都要同等对待。

例如：如果某机器有5种数据表示，4种字长，两种存储单元，则要设置 $5 \times 4 \times 2 = 40$ 种同一操作指令。

2.3.1 指令系统设计的基本原则

第 52 页

- 正交性：在指令中各个不同含义的字段，如操作类型、数据类型、寻址方式字段等，在编码时应互不相关、相互独立。
- 高效率：指指令的执行速度快、使用频度高。
- 兼容性：主要是要实现向后兼容，指令系统可以增加新指令，但不能删除指令或更改指令的功能——**指令数越来越多的原因**

英特尔处理器指令集主要包括X86、EM64T、MMX、SSE、SSE2、SSE3，SSSE3 (Super SSE3)、SSE4A、SSE4.1、SSE4.2、AVX、AVX2、AVX-512、VMX等指令集

2.3.3 指令操作码的优化

第 53 页

- 指令由两部分组成：操作码、地址码
- 指令格式的设计：确定指令字的编码方式，包括操作码字段和地址码字段的编码和表示方式。
- 指令格式的优化：如何用最短的位数来表示指令的操作信息和地址信息。

1. 哈夫曼编码

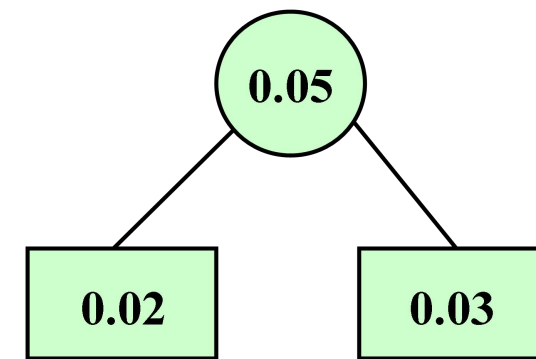
- 基本思想：当各种事件发生的概率不均等时，可以对发生概率最高的事件用最短的位数（时间）来表示（处理），而对于出现概率较低的事件，则可以用较长的位数（时间）来表示（处理），从而使总平均位数（时间）缩短。

2.3.3 指令操作码的优化

第 54 页

■ 哈夫曼编码的实现方法——构造哈夫曼树

- 将各事件按其使用频度从小到大依次排列；
- 每次从中选择两个频度值最小的结点，将其合并成一个新的结点，并把新结点画在所选结点的上面，然后用两条边把新结点分别与那两个结点相连。新结点的频度值是所选两个结点的频度值的和。
- 把新结点与其他剩余未结合的结点一起，再以上面的步骤进行处理，反复进行，直到全部结点都结合完毕、形成根结点为止。
- 操作码优化的程度可以用信息熵来衡量



画哈夫曼树的一个基本步骤

$$H = - \sum_{i=1}^n p_i \log_2 p_i$$

表示用二进制编码表示n个码点时，理论上的最短平均编码长度

2.3.3 指令操作码的优化

第 55 页

例2.1 假设某模型机有7条指令，这些指令的使用频度如表左边所示。

- (1) 计算这7条指令的操作码编码的最短平均码长；
- (2) 画出哈夫曼树，写出这7条指令的哈夫曼编码，并计算该编码的平均码长和信息冗余量。

指令	频度 p_i
I_1	0.40
I_2	0.30
I_3	0.15
I_4	0.05
I_5	0.04
I_6	0.03
I_7	0.03

解 (1)

$$H = -\sum_{i=1}^7 p_i \log_2 p_i = 2.17$$

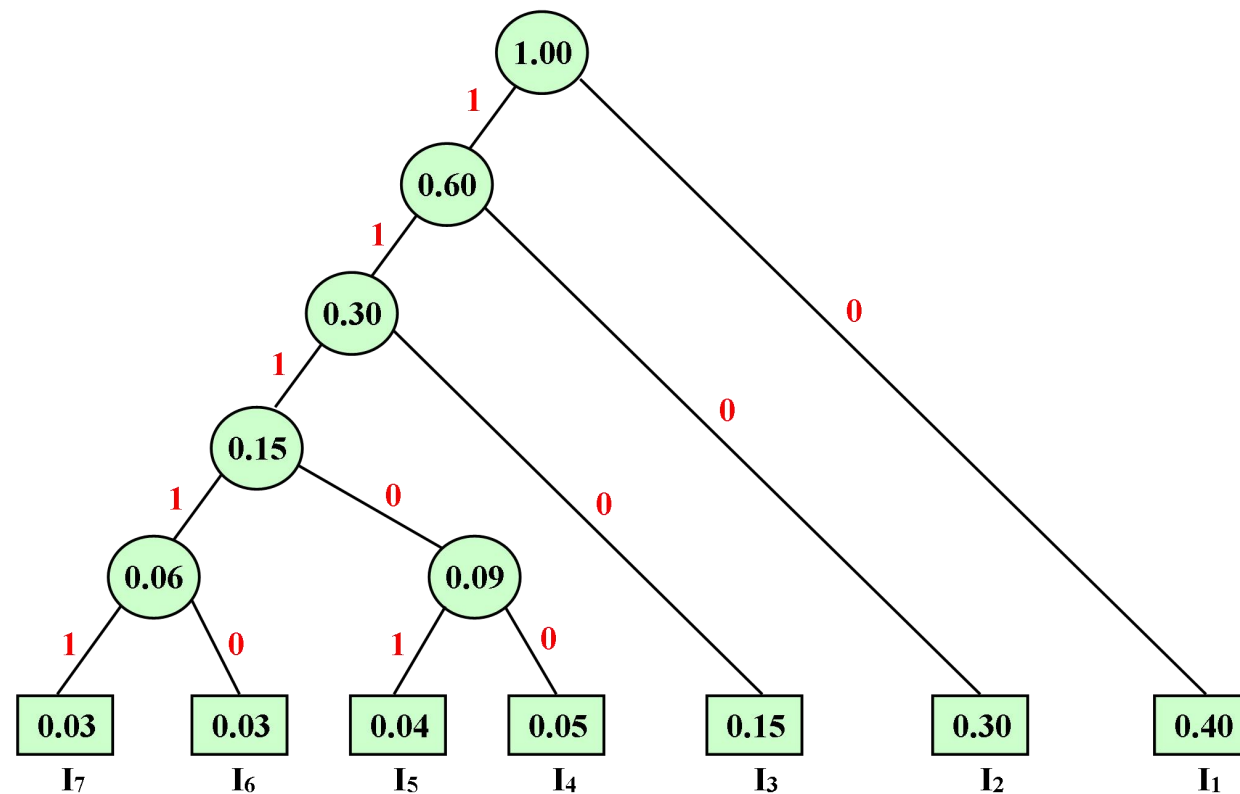
2.3.3 指令操作码的优化

第 56 页

(2) 其哈夫曼树如图所示，该树的每个叶结点分别对应于一条指令。在该树中，对每个结点向下的两个分支，分别用二进制“1”和“0”来表示。

从该哈夫曼树可以很容易地写出哈夫曼编码。

具体方法：对于任意一条指令 I_i ($i=1, 2, \dots, 7$)，从哈夫曼树根结点出发、沿一条路径连接到叶结点 I_i ，把途中所经过的各分支的“0”和“1”按从左到右的顺序记录下来，便是该指令的哈夫曼编码。上表中列出了所有指令的哈夫曼编码



哈夫曼树实例

2.3.3 指令操作码的优化

第 57 页

指令	频度 p_i	操作码使用哈夫曼编码	操作码长度 l_i
I_1	0.40	0	1
I_2	0.30	1 0	2
I_3	0.15	1 1 0	3
I_4	0.05	1 1 1 0 0	5
I_5	0.04	1 1 1 0 1	5
I_6	0.03	1 1 1 1 0	5
I_7	0.03	1 1 1 1 1	5

该哈夫曼编码的平均码长是：

$$L = \sum_{i=1}^7 p_i l_i = 2.20$$

其信息冗余量为：

$$\frac{2.20 - 2.17}{2.20} \approx 1.36\%$$

优缺点：可以减少操作码的平均位数，但所获得的编码是变长度的，不规整，不利于硬件处理。

2. 扩展操作码

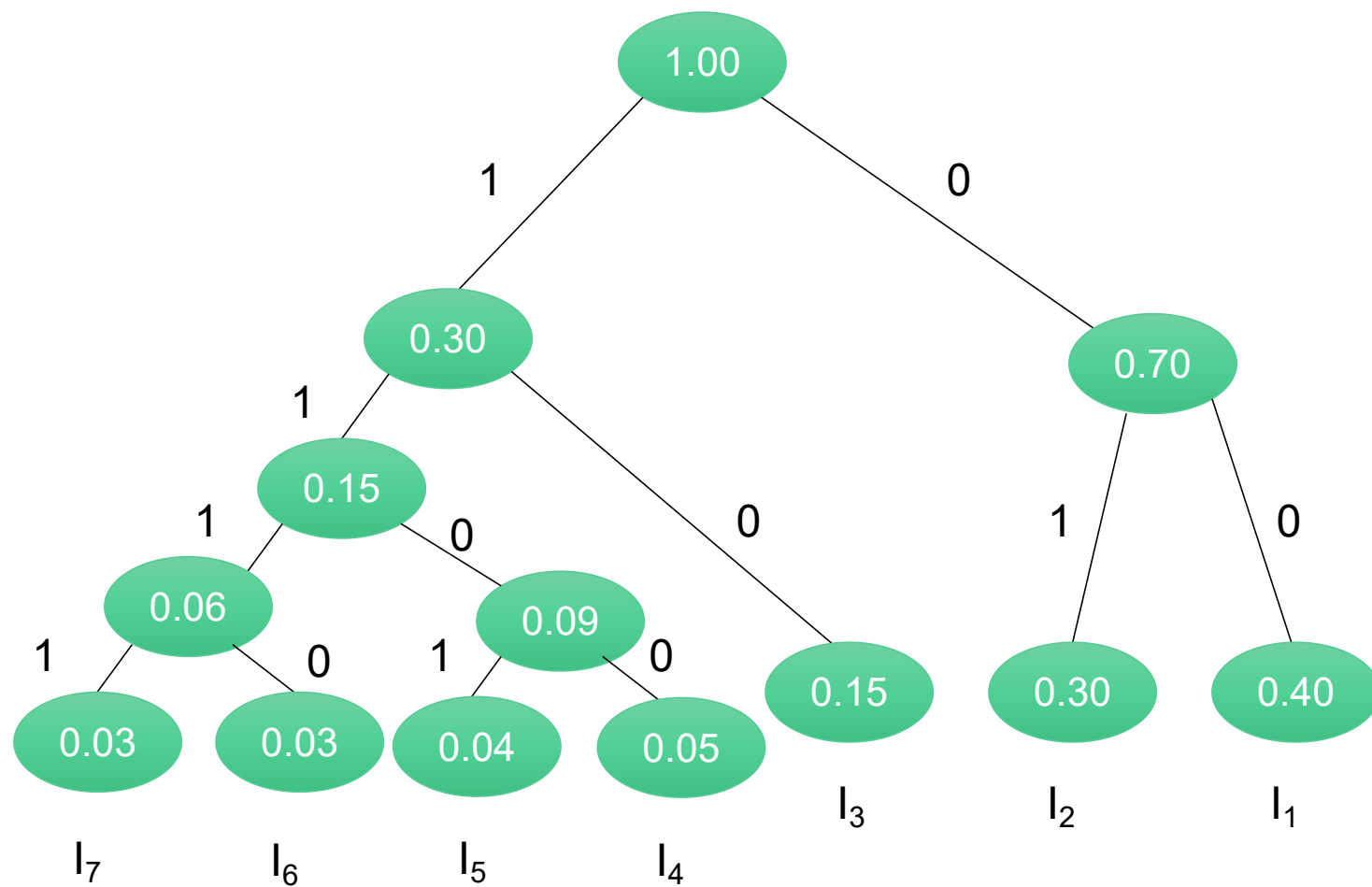
- 位于定长二进制编码和哈夫曼编码之间的一种编码方案。
- 采用有限几种固定长度的码长，仍然采用高概率的用短码、低概率用长码的哈夫曼压缩思想，使操作码平均长度缩短。
- 采用2-4的扩展操作码：用两位的00、01、10分别用于表示使用频度高的I1、I2、I3，然后用11作为高位扩展出4个4位的二进制编码，用于表示剩下的4条指令。

2.3.3 指令操作码的优化

第 59 页

指令	频度 p_i	利用哈夫曼概念的 扩展操作码	操作码 长度 l_i
I_1	0.40	0 0	2
I_2	0.30	0 1	2
I_3	0.15	1 0	2
I_4	0.05	1 1 0 0	4
I_5	0.04	1 1 0 1	4
I_6	0.03	1 1 1 0	4
I_7	0.03	1 1 1 1	4

$$\sum_{i=1}^7 p_i l_i = 2.3$$



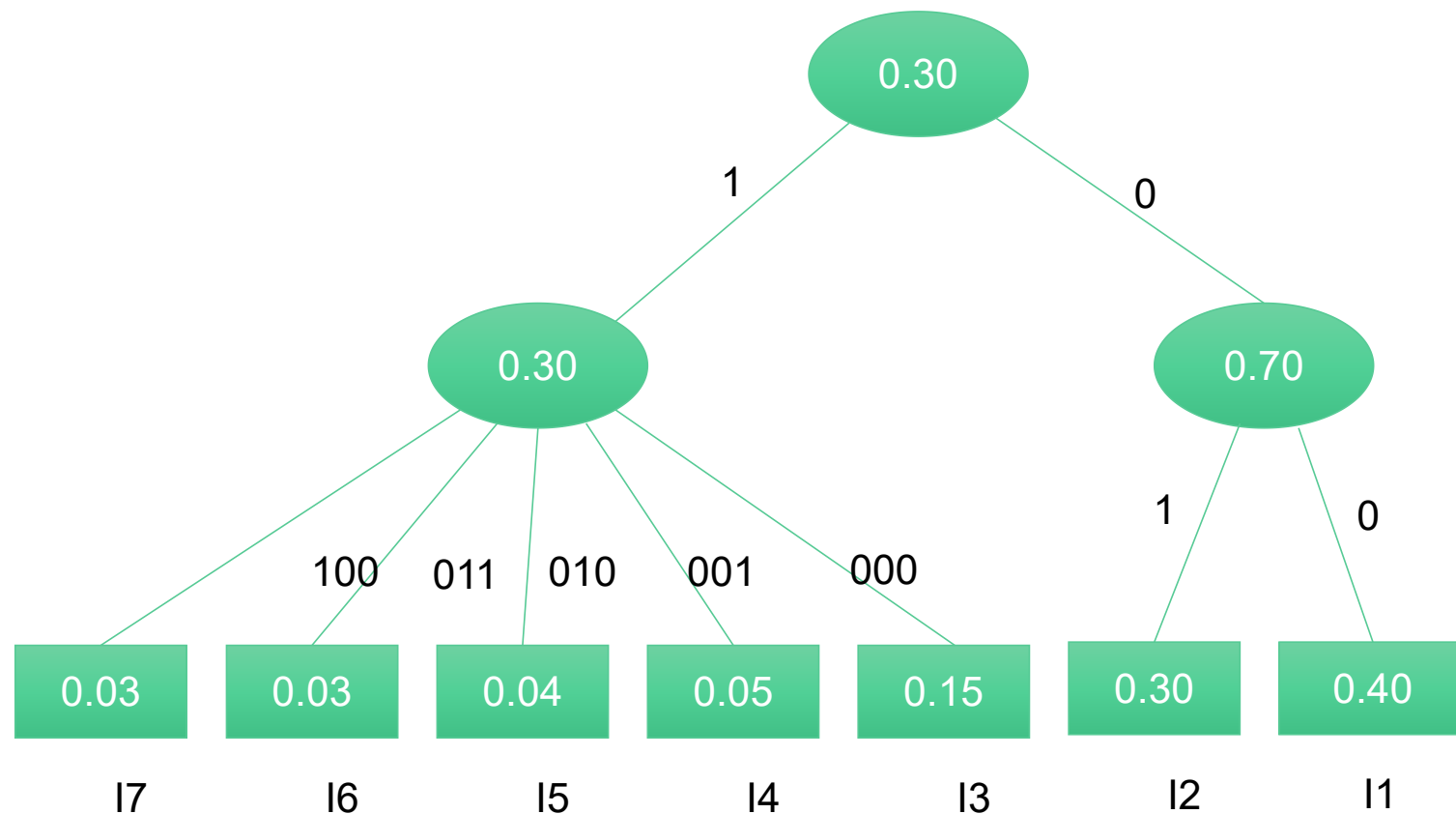
比哈夫曼编码的2.2大，但比定长的3位编码小很多。

2.3.3 指令操作码的优化

第 60 页

指令	频度 p_i	扩展操作码	操作码长度 l_i
I_1	0.40	00	2
I_2	0.30	01	2
I_3	0.15	1000	4
I_4	0.05	1001	4
I_5	0.04	1010	4
I_6	0.03	1011	4
I_7	0.03	1100	4

平均长度: $\sum_{i=1}^7 p_i l_i = 2.6$



2.3.3 指令操作码的优化

第 61 页

3. 等长扩展码

- 为了便于分级译码，一般都采用等长扩展码。在早期的计算机上，例如：15/15/15法和8/64/512法

➤ 选用哪种编码法取决于指令使用频度 p_i 的分布。若在头15种指令中 p_i 的值都比较大，但在后30种指令后急剧减少，则应选择15/15/15法；若 p_i 的值在头8种指令中较大，之后的64种指令的 p_i 值也不太低，则应选择8/64/512法。

➤ 衡量标准：看哪种编码法能使平均码长最短。

15 {
0000
0001
⋮
1110
1111 0000
1111 0001
⋮
1111 1110
1111 1111 0000
1111 1111 0001
⋮
1111 1111 1110

15/15/15 编码法

8 {
0 000
0 001
⋮
0 111
64 {
1 000 0 000
1 000 0 001
⋮
1 111 0 111
512 {
1 000 1 000 0 000
1 000 1 000 0 001
⋮
1 111 1 111 0 111

8/64/512 编码法

2.3.3 指令操作码的优化

第 62 页

4. 定长操作码

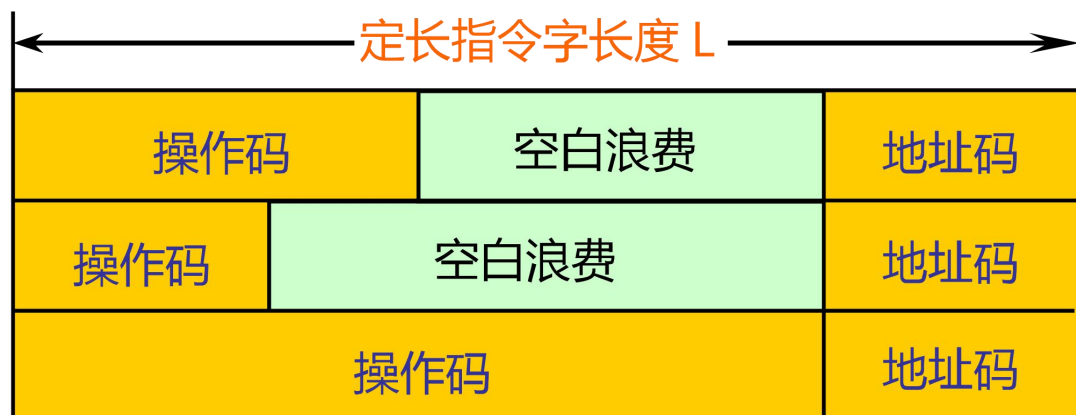
- 固定长度的操作码：所有指令的操作码都是同一的长度（如8位）。许多计算机都采用（特别是RISC结构的计算机）
- 保证操作码的译码速度、减少译码的复杂度。
- 以程序的存储空间为代价来换取硬件实现上的好处。

0001100	rs2	rs1	rm	rd	101001	R fdiv.s
0001100	00000	rs1	rm	rd	101001	R fsqrt.s
0010000	rs2	rs1	000	rd	101001	R fsgnj.s
0010000	rs2	rs1	001	rd	101001	R fsgnjn.s
0010000	rs2	rs1	010	rd	101001	R fsgnjx.s
0010100	rs2	rs1	000	rd	101001	R fmin.s

2.3.4 指令字格式的优化

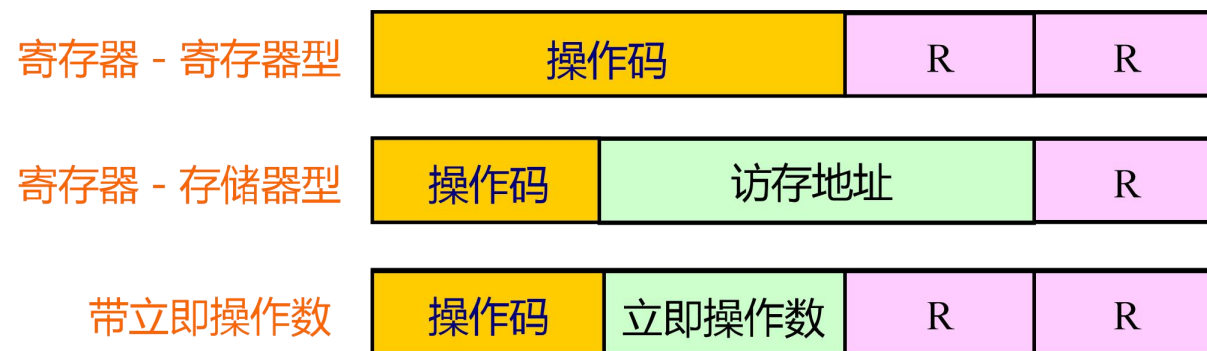
第 63 页

1. 如果指令字的宽度固定，地址码的长度和个数固定，则操作码的缩短并不能带来好处，只是使指令字中出现空白浪费。



2. 采用地址个数可变和/或地址码长度可变的方案

- 利用操作码缩短所带来的好处；最常用的操作码最短，其地址字段个数最多。能够使指令的功能增强，从总体上减少所需指令条数。



2.3.4 指令字格式的优化

第 64 页

3. 考虑因素

- 计算机中寄存器的个数和寻址方式的数目对机器的指令字长有很大的影响；
- 指令字的平均长度增加了，程序的平均长度也就增加了；
- 在指令系统的设计中，要在指令字长与寄存器的个数以及寻址方式的个数之间进行折中。

4. 指令系统的3种编码格式： 可变长度编码格式、固定长度编码格式、混合型编码格式

- 可变长度编码格式
 - ✓ 当指令系统的寻址方式和操作种类很多时，这种编码格式是最好的。
 - ✓ 用最少的二进制位来表示目标代码。可能会使各条指令的字长和执行时间相差很大。

操作码	地址描述符 1	地址码 1	...	地址描述符 n	地址码 n
-----	---------	-------	-----	---------	-------

2.3.4 指令字格式的优化

第 65 页

➤ 固定长度编码格式

- ✓ 将操作类型和寻址方式一起编码到操作码中。当寻址方式和操作类型非常少时，这种编码格式非常好。
- ✓ 可以有效降低译码的复杂度，提高译码的速度。大部分RISC的指令系统均采用这种编码格式。

操作码	地址码 1	地址码 2	地址码 3
-----	-------	-------	-------

➤ 混合型编码格式

- ✓ 提供若干种固定的指令字长。
- ✓ 以期达到既能够减少目标代码长度又能降低译码复杂度的目标。

操作码	地址描述符	地址码
-----	-------	-----

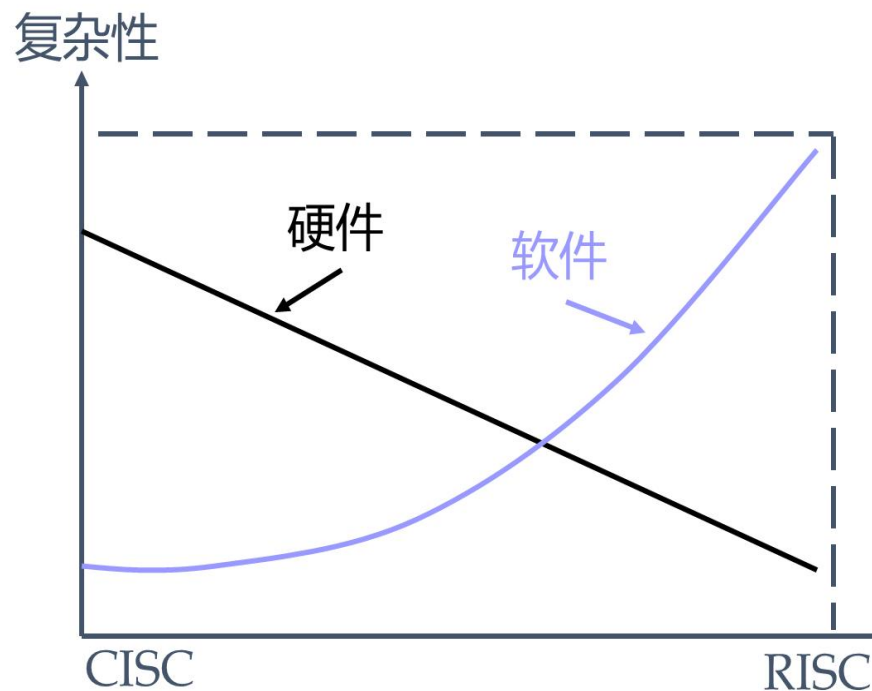
操作码	地址描述符 1	地址描述符 2	地址码
-----	---------	---------	-----

操作码	地址描述符	地址码 1	地址码 2
-----	-------	-------	-------

2.4 指令系统的发展和改进

2.4.1 沿CISC方向发展和改进指令系统 第 67 页

复杂而高效



简约而不简单

思路

特点

应用

CISC体系

为了便于软件编程和提高程序运行速度，增加可实现复杂功能的指令和多种灵活的编址方式

- 1) 编译优化、程序代码量低
- 2) 指令执行效率高
- 3) 处理器性能高

通用计算机系统
(PC、笔记本、服务器、集群、巨型机等)

RISC体系

为了便于硬件实现和降低处理器实现成本，只实现使用频率很高的少数指令与规则的编址方式

- 1) 易于硬件实现
- 2) 易于流水实现与并行处理
- 3) 指令与数据存储效率高

嵌入式计算机系统
(手机、工业控制、医疗电子)

2.4.1 沿CISC方向发展和改进指令系统 第 68 页

1. CISC指令系统的一大特点： 指令数量多、功能多样
2. 增强指令功能主要是从以下3个方面着手：
 - 面向目标程序增强指令功能
 - 面向高级语言的优化实现来改进指令系统
 - 面向操作系统的优化实现改进指令系统

2.4.1 沿CISC方向发展和改进指令系统 第 69 页

3. 面向目标程序增强指令功能

- 对大量的目标程序及其执行情况进行统计分析，找出那些使用频度高、执行时间长的指令或指令串。对于使用频度高的指令，用硬件加快其执行；对于使用频度高的指令串，用一条新的指令来替代。既能减少目标程序的执行时间，也能有效地缩短程序的长度。
- 改进方法： 增强运算型指令的功能、增强数据传送指令的功能和增强程序控制指令的功能

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \quad \text{矩阵 } A \times B = C$$

□方案一：两个数相乘 $A \times B = C$

mul R_A, R_B, R_C



□方案二：三个数相乘加 $A \times B + C = D$

mac R_A, R_B, R_C, R_D

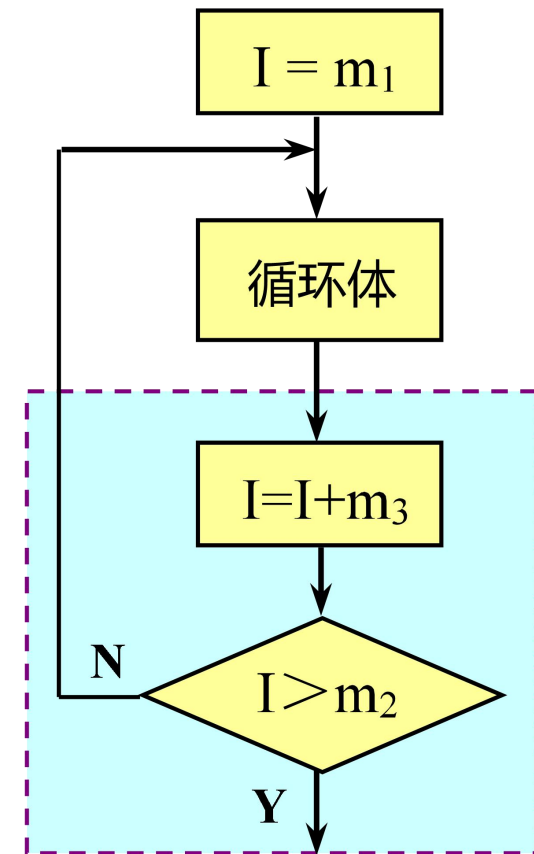
28

24

2.4.1 沿CISC方向发展和改进指令系统 第 70 页

例如：循环在程序中占有相当大的比例，所以在指令上提供专门的支持。

- 改进前：循环控制部分通常用3条指令完成：
 - 一条加法指令
 - 一条比较指令
 - 一条分支指令
- 改进后：设置循环控制指令，用一条指令完成上述3条指令的功能。



一般循环程序的结构

2.4.1 沿CISC方向发展和改进指令系统 第 71 页

4. 面向高级语言的优化实现来改进指令系统——缩小高级语言与机器语言的语义差距

高级语言与一般的机器语言的语义差距非常大，为高级语言程序的编译带来了一些问题。

- (1) 编译器本身比较复杂；
- (2) 编译生成的目标代码比较难以达到很好的优化。

d=a*b+c
//C语言

编译

mul R0, R1, R2 ; // 乘法指令
add R2, R3, R4; // 加法指令

优化

mac R0, R1, R2, R3 ;
// 乘加指令

2.4.1 沿CISC方向发展和改进指令系统 第 72 页

增强对高级语言和编译器的支持

(1) 对高级语言中使用频度高、执行时间长的语句，增强有关指令的功能，加快这些指令的执行速度，或者增加专门的指令，可以达到减少目标程序的执行时间和减少目标程序长度的目的。

(2) 增强系统结构的规整性，减少系统结构中的各种例外情况。（面向高级语言的计算机）

① 间接执行高级语言机器：高级语言作为机器的汇编语言。

② 直接执行高级语言的机器：直接把高级语言作为机器语言。（一种比较激进的方法）

d=a*b+c
//C语言

增强

mac R0, R1, R2, R3 ;
// 乘加指令

激进

d=a*b+c
//高级语言作为机器语言

2.4.1 沿CISC方向发展和改进指令系统 第 73 页

5. 面向操作系统的优化实现改进指令系统

- 操作系统和计算机系统结构是紧密联系的，操作系统的实现在很大程度上取决于系统结构的支持。
- 指令系统对操作系统的支持主要有：
 - ✓ 处理机工作状态和访问方式的切换；
 - ✓ 进程的管理和切换；
 - ✓ 存储管理和信息保护；
 - ✓ 进程的同步与互斥，信号灯的管理等。

支持操作系统的有些指令属于特权指令，一般用户程序是不能使用的。

2.4.2 沿RISC方向发展和改进指令系统 第 74 页

1.CISC指令系统结构存在的问题（1979年开始，Patterson等人的研究）

- 各种指令的使用频度相差悬殊，许多指令很少用。
 - 据统计：只有20%的指令使用频度比较高，占运行时间的80%。而其余80%的指令只在20%的运行时间内才会用到。使用频度高的指令也是最简单的指令。
 - 指令系统庞大，指令条数很多，许多指令的功能又很复杂，使得控制器硬件非常复杂

2.4.2 沿RISC方向发展和改进指令系统 第 75 页

1.CISC指令系统结构存在的问题（1979年开始，Patterson等人的研究）

■ 导致的问题：

- 占用了大量的芯片面积（如占用CPU芯片总面积的一半以上），给VLSI设计造成很大的困难；增加了研制时间和成本，容易造成设计错误。
- 许多指令由于操作繁杂，其CPI值比较大，执行速度慢。采用这些复杂指令有可能使整个程序的执行时间反而增加。
- 由于指令功能复杂，规整性不好，不利于采用流水技术来提高性能。

2.4.2 沿RISC方向发展和改进指令系统 第 76 页

Intel 80x86最常用的10条指令

执行频度排序	80x86指令	指令执行频度（占执行指令总数的百分比）
1	load	22%
2	条件分支	20%
3	比较	16%
4	store	12%
5	加	8%
6	与	6%
7	减	5%
8	寄存器 - 寄存器间数据移动	4%
9	调用子程序	1%
10	返回	1%
合 计		95%

2.4.2 沿RISC方向发展和改进指令系统 第 77 页

2. 设计RISC机器遵循的原则，指令条数少、指令功能简单，方法如下：

- 指令系统：选取使用频度高的指令，在此基础上补充一些最有用的指令；
- 寻址机制：采用简单又统一的指令格式，并减少寻址方式；一般来说，指令字长都为32位或64位；
- 执行机制：采用流水线机制，指令的执行在单个机器周期内完成；
- 访存机制：采用load-store结构，只有load和store指令才能访问存储器，其它指令的操作都是在寄存器之间进行；
- 实现机制：大多数指令都采用硬连逻辑来实现；
- 编译机制：强调优化编译器的作用，为高级语言程序生成优化的代码。

2.4.2 沿RISC方向发展和改进指令系统 第 78 页

3. 早期的RISC微处理器

- 1981年，Berkeley分校的Patterson 等人的32位微处理器RISC I
 - 31条指令，指令字长都是32位，78个通用寄存器，时钟频率为8MHz；
 - 控制部分所占的芯片面积只有约6%。商品化微处理器MC68000和Z8000分别为50%和53%；性能比MC68000和Z8000快3 ~ 4倍。
- 1983年的RISC II
 - 指令条数为39，通用寄存器个数为138，时钟频率为12MHz。
 - 后来发展成了Sun公司的SPARC系列微处理器。
- 1981年，Stanford大学Hennessy等人的MIPS后来发展成了MIPS Rxxx系列微处理器
- 2010年加州大学伯克利分校研发的RISC-V指令集。

2.5 操作数类型和大小

2.5.1 数据类型和表示

第 80 页

■ 系统结构设计者首先要解决的问题：数据类型与表示

- 数据类型和表示是软硬件主要界面之一。
- 数据类型：面向应用、面向软件系统所处理的各种数据结构。例如定点、浮点、十进制、字符、字符串、逻辑、堆栈和向量等。
- 数据表示：硬件结构能够识别、指令系统可以直接调用的那些结构。

系统结构设计者要解决的问题：**如何确定数据类型与表示？**

(软硬件取舍折衷的问题)

2.5.1 数据类型和表示

第 81 页

■ 确定数据类型和数据表示的原则：

- 一是缩短程序的运行时间；
- 二是减少CPU与主存储器之间的通信量；
- 三是这种数据表示的通用性和利用率。

数据类型和数据表示在不断发展， 例如矩阵、树、图、表及自定义数据类型等。

2.5.1 数据类型和表示

第 82 页

例2.2：计算 $C = A + B$ ，其中， A 、 B 、 C 均为 200×200 的矩阵，分析采用向量数据类型的作用。

解：如果在没有向量数据的计算机上实现，一般需要6条指令，其中有4条指令要循环4万次。因此，CPU与主存储器之间的通信量：

取指令： $2 + 4 \times 40,000$ 条，

读或写数据： $3 \times 40,000$ 个，

共要访问主存储器： $7 \times 40,000$ 次以上

如果有向量数据类型，只需要一条指令。

减少访问主存（取指令）次数 $4 \times 40,000$ 次

用软件和硬件结合的方法实现新的数据

2.5.1 数据类型和表示

第 83 页

- 操作数表示方法一：操作数的类型由指令中的操作码指定（最常见的一种方法）

MIPS Instruction Set (Part 1)

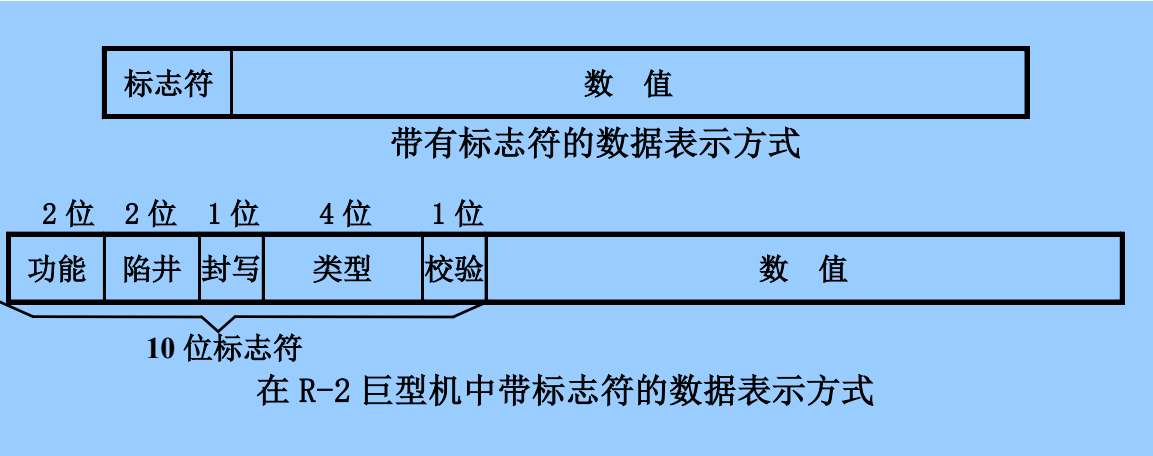
Instruction	Meaning
<i>Arithmetic</i>	
add	integer addition
subtract	integer subtraction
add immediate	integer addition (register + constant)
add unsigned	unsigned integer addition
subtract unsigned	unsigned integer subtraction
add immediate unsigned	unsigned addition with a constant
move from coprocessor	access coprocessor register
multiply	integer multiplication

MIPS Floating Point Instructions

Instruction	Meaning
<i>Arithmetic</i>	
FP add	floating point addition
FP subtract	floating point subtraction
FP multiply	floating point multiplication
FP divide	floating point division
FP add double	double-precision addition
FP subtract double	double-precision subtraction
FP multiply double	double-precision multiplication
FP divide double	double-precision division

2.5.1 数据类型和表示

■ 操作数表示方法二：数据可以附上由硬件解释的标记，由这些标记符指定操作数类型，从而选择适当的运算。



带标志符的数据表示法

- 在B5000大型机中，每个数据有一位标志符
- 在B6500和B7500大型机中，每个数据有三位来标志符
- 在R-2巨型机中采用10位标志符

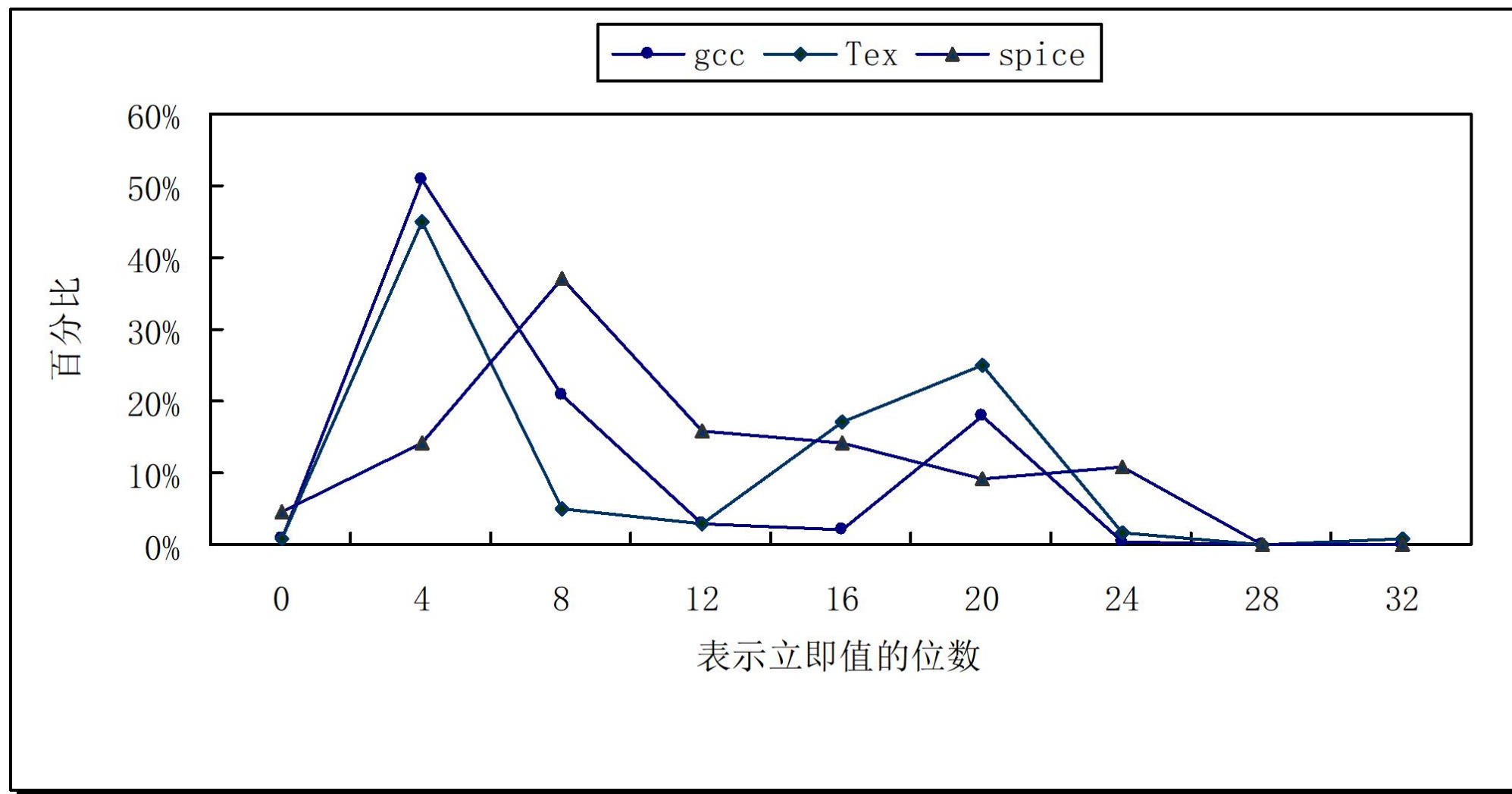
- 优点：简化指令系统，可由硬件自动实现一致性检查和类型转换，缩小了机器语言与高级语言的语义差距，简化编译器等。
- 缺点：由于需要在执行过程中动态检测标志符，动态开销比较大，所以采用这种方案的机器很少见。

2.5.1 数据类型和表示

第 85 页

- 操作数的大小：操作数的位数或字节数，主要包括：字节（8位）、半字（16位）、字（32位）、双字（64位）
 - 字符：用ASCII码表示，为一个字节大小。
 - 整数：用二进制补码表示，其大小可以是字节、半字或单字。
 - 浮点操作数：单精度浮点数（1个字）、双精度浮点数（双字），一般都采用IEEE 754浮点标准
 - 十进制操作数类型
 - ✓ 压缩十进制或二进制编码十进制（BCD码）：用4位二进制编码表示数字0～9，并将两个十进制数字合并到一个字节中存储。
 - ✓ 非压缩十进制：将十进制数直接用字符串来表示。

2.5.1 数据类型和表示



2.5.1 数据类型和表示

■ 访问不同操作数大小的频率（SPEC基准程序）

操作数大小	访问频度	
	整型平均	浮点平均
字节	7%	0%
半字	19%	0%
单字	74%	31%
双字	0%	69%

一台32位的机器应该支持8、16、32位整型操作数以及32位和64位的IEEE 754标准浮点操作数。

■ 确定方法（软硬件取舍折中的问题）：

- 所有数据类型中最常用、相对比较简单、用硬件实现比较容易的几种。
- 研究这些数据类型的逻辑结构与物理结构之间的关系，并给出相应的算法。

指令系统通常有上百条或几百条指令，从它们所完成的功能来看，一个较为完善的指令系统，应具备以下各类指令：

1、数据传送类指令

这类指令的功能是实现寄存器与寄存器，寄存器与存储单元以及存储单元与存储单元之间的数据传送

数据传送指令主要包括取数指令、存数指令、传送指令、成组传送指令、字节交换指令、清累加器指令、堆栈操作指令等等。

2、算术运算指令

这类指令包括二进制定点加、减、乘、除指令，浮点加、减、乘、除指令，求反、求补指令，算术移位指令，算术比较指令，十进制加、减运算指令等。这类指令主要用于定点或浮点的算术运算。

3、逻辑运算指令

这类指令包括逻辑加、逻辑乘、逻辑比较、测等指令、按位加、逻辑移位等指令，主要用于无符号数的位操作、代码的转换、判断及运算。移位指令用来对寄存器的内容实现左移、右移或循环移位。

4、程序控制指令

程序控制指令也称**转移指令**。执行程序时，有时机器执行到某条指令时，出现了几种不同结果，这时机器必须执行一条转移指令，根据不同结果进行转移，从而改变程序原来执行的顺序。这种转移指令称为条件转移指令。

除各种条件转移指令外，还有**无条件转移指令**、**转子程序指令**、**返回主程序指令**、**中断返回指令**等。转移指令的转移地址一般采用直接寻址和相对寻址方式来确定。

2.6 典型指令系统 (RISC-V)

2.6 典型指令系统结构

第 92 页

RISC-V是一个基于精简指令集（RISC）原则的**开源**指令集架构（ISA）



ABOUT ▾ MEMBERSHIP ▾ SPECS & SUPPORT ▾ CORES & TOOLS ▾ NEWS ▾ EVENTS ▾



Blog

🏠 / EE Times China Report: Open Source Chips And Agile Development For Next-Generation Computing

完全开源

架构简单

支持linux

模块化

工具完整

2010年始于加州大学伯克利分校，许多贡献者是该大学以外的志愿者和行业工作者。RISC-V指令集的设计充分考虑了小型、快速、低功耗的应用需求，对特定的微架构进行了简洁优化。

2.6 典型指令系统结构

第 93 页

基本指令集

RV32I
(47条)

32 位地址空间与整数指令，
支持 32 个通用整数寄存器

RV32E
(47条)

RV32I 的子集，仅支持 16
个通用整数寄存器

RV64I
(59条)

64 位地址空间与整数指令
及一部分 32 位的整数指令

RV128I
(71条)

128 位地址空间与整数指令
及一部分 64 位和 32 位的
指令

扩展指令集

M
(8条)

整数乘法与除法指令

A
(11条)

存储器原子 (Atomic) 操
作指令 (load/store)

F
(26条)

单精度 (32 比特) 浮点指
令

D
(26条)

双精度 (64 比特) 浮点指
令，必须支持 F 扩展指

C
(46)

压缩指令，指令长度为 16
位

规则

简洁

高效

优雅

RV32IMAFDC: RV32GC



2.6 典型指令系统结构

第 94 页

完备性

- 使用指令系统提供的指令足够直接编写各种程序;
- 完备性要求指令系统丰富、功能齐全、编程方便

规则性

- 对称性: 所有指令均可用各种寻址方式访问寄存器和存储器;
- 匀齐性: 任一操作指令可支持各种数据类型; 格式一致性: 指令长度和数据长度均一致, 便于处理与存取

有效性

- 使用指令系统编写的程序能够高效地运行;
- 主要表现在程序占据存储空间小、执行速度快

兼容性

- 向上兼容——低档机上运行的软件可以在高档机上运行

2.6 典型指令系统结构

完整性： 在一个有限可用的存储空间内，对于任何可解的问题，编制计算程序时，指令系统所提供的指令足够使用

操作类型	实 例
算术和逻辑运算	算术运算和逻辑操作：加，减，乘，除，与，或等
数据传输	load, store
控制	分支，跳转，过程调用和返回，自陷等
系统	操作系统调用，存储器管理等
浮点	浮点操作：加，减，乘，除，比较等
十进制	十进制加，十进制乘，十进制到字符的转换等
字符串	字符串移动，字符串比较，字符串搜索等
图形	像素操作，压缩/解压操作等

2.6 典型指令系统结构

第 96 页

整数计算

add { immediate } subtract { - }

load_upper_immediate

add_upper_immediate_to_pc

运算

控制转移

存取操作

混杂操作

$\left\{ \begin{array}{l} \underline{\text{and}} \\ \underline{\text{or}} \\ \underline{\text{exclusive or}} \end{array} \right\} \{ \underline{\text{immediate}} \}$

$\left\{ \begin{array}{l} \underline{\text{shift_left_logical}} \\ \underline{\text{shift_right_arithmetic}} \\ \underline{\text{shift_right_logical}} \end{array} \right\} \{ \underline{\text{immediate}} \}$

逻辑

set_less_than { immediate } { unsigned } 比较

2.6 典型指令系统结构

第 97 页

整数计算

控制转移

存取操作

混杂操作

$\underline{\text{branch}} \left\{ \begin{array}{l} \underline{\text{equal}} \\ \underline{\text{no_equal}} \end{array} \right\}$

分支

$\underline{\text{branch}} \left\{ \begin{array}{l} \underline{\text{great than or equal}} \\ \underline{\text{less than}} \end{array} \right\} \left\{ \begin{array}{l} \underline{\text{signed}} \\ \underline{\text{unsigned}} \end{array} \right\}$

$\underline{\text{jump_and_link}} \left\{ \begin{array}{l} \underline{\text{register}} \end{array} \right\}$

跳转

2.6 典型指令系统结构

第 98 页

整数计算

控制转移

存取操作

混杂操作

$$\underline{\text{load}} \left\{ \begin{array}{c} \text{byte} \\ \underline{\text{halfword}} \\ \underline{\text{word}} \end{array} \right\}$$
$$\underline{\text{load}} \left\{ \begin{array}{c} \text{byte} \\ \underline{\text{halfword}} \end{array} \right\} \underline{\text{unsigned}}$$

取
数

$$\underline{\text{store}} \left\{ \begin{array}{c} \underline{\text{byte}} \\ \underline{\text{halfword}} \\ \underline{\text{word}} \end{array} \right\}$$

存
数

2.6 典型指令系统结构

第 99 页

整数计算

fence loads & stores

fence instruction & data

同步

控制转移

environment $\left\{ \begin{array}{l} \text{break} \\ \text{call} \end{array} \right\}$

环境

存取操作

control status register $\left\{ \begin{array}{l} \text{read \& clear bit} \\ \text{read \& set bit} \\ \text{read \& write} \end{array} \right\} \left\{ \begin{array}{l} \text{immediate} \end{array} \right\}$

控制状态

混杂操作

2.6 典型指令系统结构

第 100 页

规则性：主要包括对称性、均匀性与一致性

■ **对称性：**所有与指令系统有关的存储单元的使用、操作码的设置等都是对称的。

例如：在存储单元的使用上，所有通用寄存器都要同等对待。在操作码的设置上，如果设置了A - B的指令，就应该也设置B - A的指令。

add rd, rs1, rs2  $f[rd] = f[rs1] + f[rs2]$

sub rd, rs1, rs2  $f[rd] = f[rs1] - f[rs2]$

2.6 典型指令系统结构

第 101 页

规则性：主要包括对称性、均匀性与一致性

■ **均匀性：**指对于各种不同的操作数类型、字长、操作种类和数据存储单元，指令的设置都要同等对待。

例如：如果某机器有5种数据表示，4种字长，两种存储单元，则要设置 $5 \times 4 \times 2 = 40$ 种同一操作的指令。



2.6 典型指令系统结构

第 102 页

规则性：主要包括对称性、均匀性与一致性

■ **一致性：**指令长度和数据长度均一致，使于处理与存存取。

RISC-V基本指令集

RV32I
(47条)

32 位地址空间与整数指令， 支持
32 个通用整数寄存器

RV64I
(59条)

64 位地址空间与整数指令及一部分
32 位的整数指令

RV32E
(47条)

RV32I 的子集， 仅支持 16 个通用整
数寄存器

RV128I
(71条)

128 位地址空间与整数指令及一部分
64 位和 32 位的指令

2.6 典型指令系统结构

第 103 页

有效性：主要包括正交性和高效率

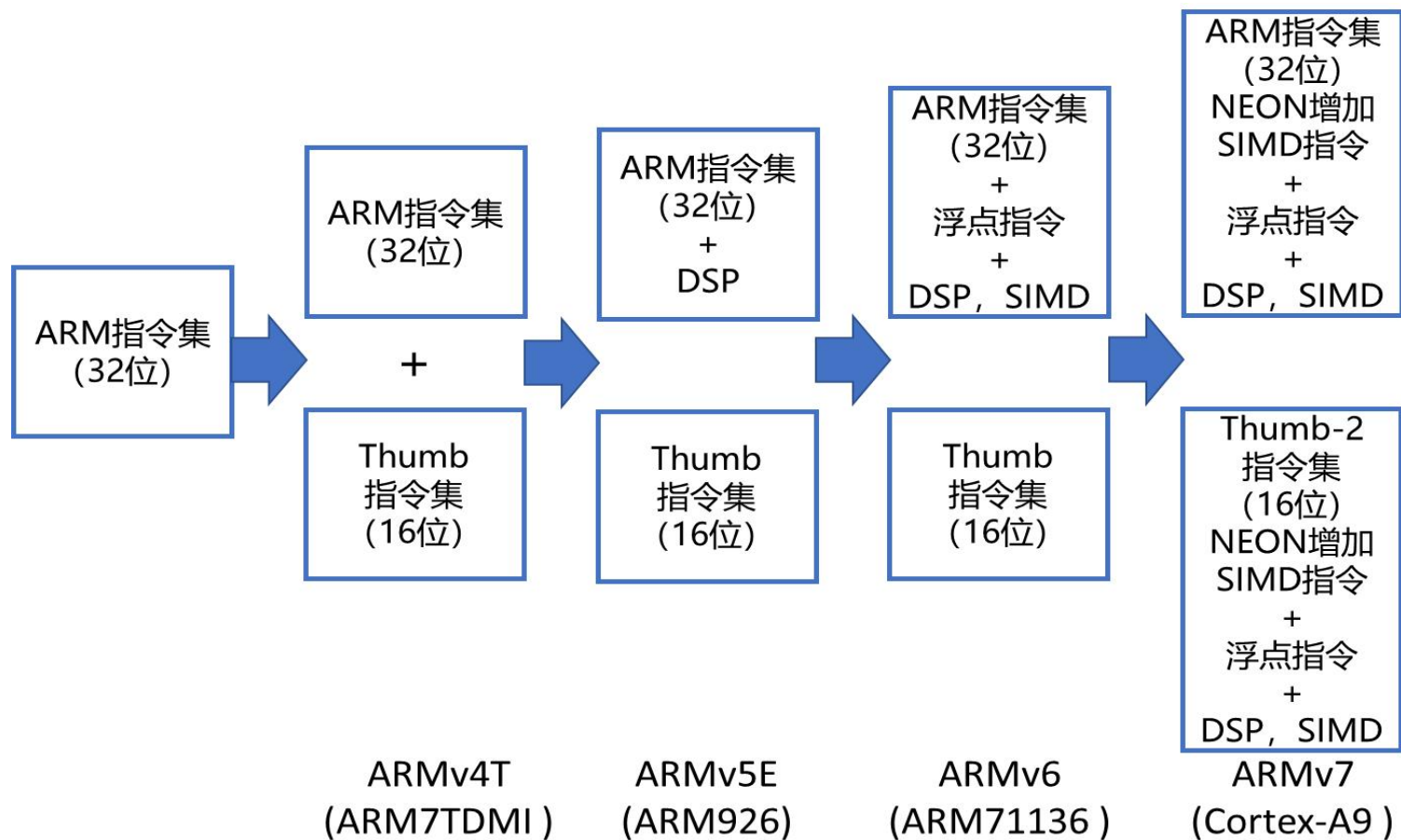
- **正交性：**在指令中各个不同含义的字段，如操作类型、数据类型、寻址方式字段等，在编码时应互不相关、相互独立。
- **均匀性：**指令的执行速度快、使用频度高。

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0				
funct7				rs2			rs1		funct3		rd			opcode		R-type		
imm[11:0]						rs1		funct3		rd			opcode		I-type			
imm[11:5]				rs2			rs1		funct3		imm[4:0]			opcode		S-type		
imm[12]		imm[10:5]			rs2			rs1		funct3		imm[4:1]		imm[11]		opcode		B-type
imm[31:12]										rd			opcode			U-type		
imm[20]		imm[10:1]				imm[11]		imm[19:12]				rd			opcode		J-type	

2.6 典型指令系统结构

第 104 页

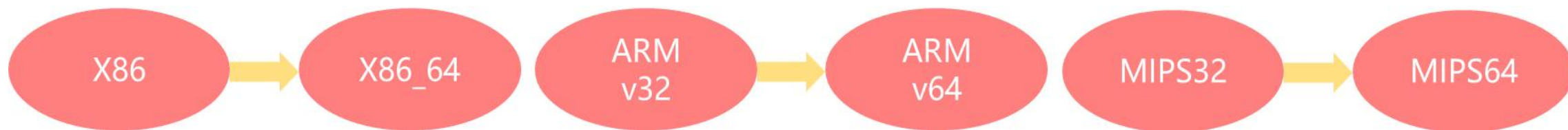
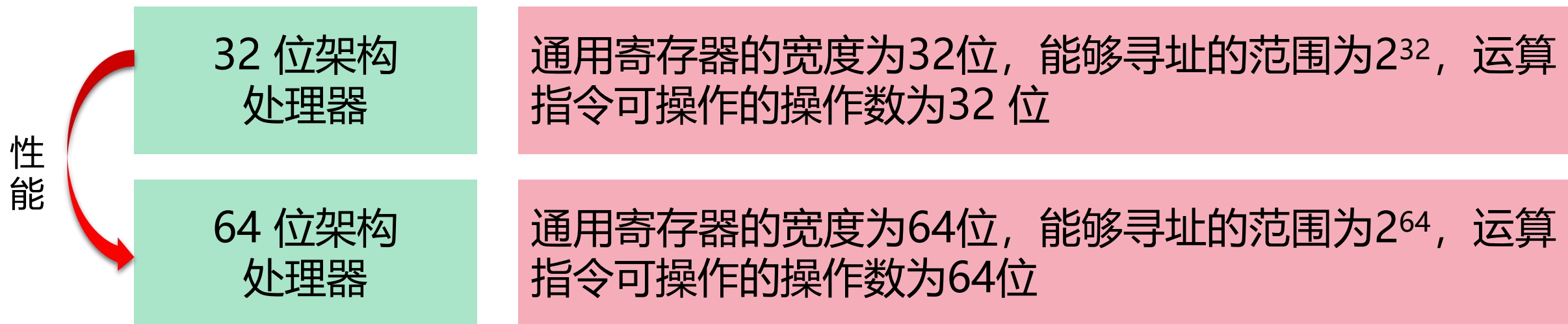
兼容性：主要是要实现向后兼容，指令系统可以增加新指令，但不能删除指令或更改指令的功能



2.6 典型指令系统结构

第 105 页

处理器架构的位数指通用寄存器的宽度，决定了寻址范围的大小、数据运算能力的强弱



2.6 典型指令系统结构

第 106 页

随着性能需求的不断增加，处理器指令集架构宽度越来越大，但是否意味着性能越来越好？

基本指令集

RV32I
(47条)

32位地址空间与整数指令，
支持32个通用整数寄存器

RV64I
(59条)

64位地址空间与整数指令及
部分 32 位的整数指令

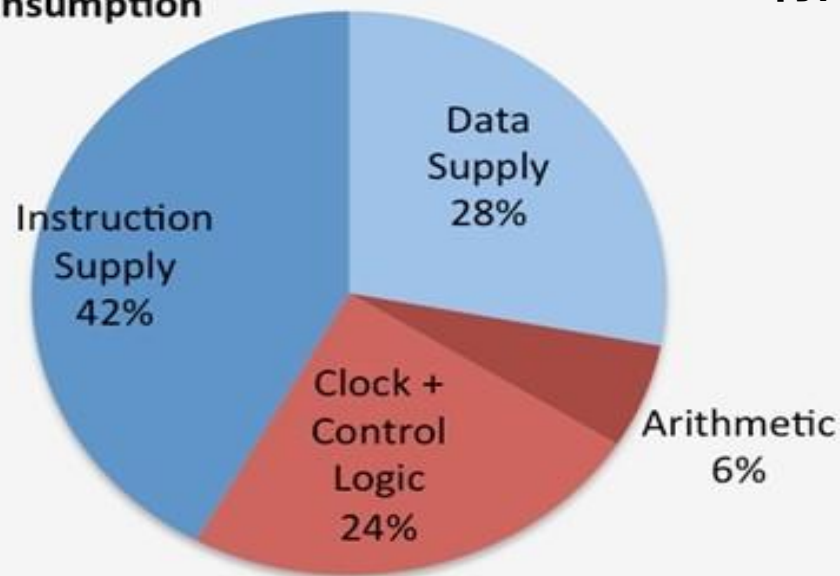
RV128I
(71条)

128位地址空间与整数指令
及部分64位和32位的指令

性能越来越高

Embedded Processor
Energy Consumption

45nm CMOS制程



现象1：存取指令所消耗的能量比典型运算的能耗大

2.6 典型指令系统结构

第 107 页

代码密度：编译具有同样功能的一段代码，指令与数据占用的空间越大，则代码密度低，也就是效率低；反之，代码密度高，执行效率高。

```
void insertion_sort(long a[], size_t n)
{
    for (size_t i = 1, j; i < n; i++) {
        long x = a[i];
        for (j = i; j > 0 && a[j-1] > x; j--) {
            a[j] = a[j-1];
        }
        a[j] = x;
    }
}
```

插入排序 (Insertion sort) 是一种简单直观且稳定的排序算法。如果有一个已经有序的数据序列，要求在这个已经排好的数据序列中插入一个数，但要求插入后此数据序列仍然有序

2.6 典型指令系统结构

第 108 页

```
# RV32I (19 instructions, 76 bytes, or 52 bytes with RVC)
# a1 is n, a3 points to a[0], a4 is i, a5 is j, a6 is x
0: 00450693 addi a3,a0,4 # a3 is pointer to a[i]
4: 00100713 addi a4,x0,1 # i = 1
Outer Loop:
8: 00b76463 bltu a4,a1,10 # if i < n, jump to Continue Outer loop
Exit Outer Loop:
c: 00008067 jalr x0,x1,0 # return from function
Continue Outer Loop:
10: 0006a803 lw a6,0(a3) # x = a[i]
14: 00068613 addi a2,a3,0 # a2 is pointer to a[j]
18: 00070793 addi a5,a4,0 # j = i
Inner Loop:
1c: ffc62883 lw a7,-4(a2) # a7 = a[j-1]
20: 01185a63 bge a6,a7,34 # if a[j-1] <= a[i], jump to Exit Inner Loop
24: 01162023 sw a7,0(a2) # a[j] = a[j-1]
28: fff78793 addi a5,a5,-1 # j--
2c: ffc60613 addi a2,a2,-4 # decrement a2 to point to a[j]
30: fe0796e3 bne a5,x0,1c # if j != 0, jump to Inner Loop
Exit Inner Loop:
34: 00279793 slli a5,a5,0x2 # multiply a5 by 4
38: 00f507b3 add a5,a0,a5 # a5 is now byte address of a[j]
3c: 0107a023 sw a6,0(a5) # a[j] = x
40: 00170713 addi a4,a4,1 # i++
44: 00468693 addi a3,a3,4 # increment a3 to point to a[i]
48: fc1ff06f jal x0,8 # jump to Outer Loop
```

RV32I程序

19个指令
76个字节

2.6 典型指令系统结构

第 109 页

```
# ARM-32 (19 instructions, 76 bytes; or 18 insns/46 bytes with Thumb-2)
# r0 points to a[0], r1 is n, r2 is j, r3 is i, r4 is x
0: e3a03001 mov  r3, #1          # i = 1
4: e1530001 cmp  r3, r1          # i vs. n (unnecessary?)
8: e1a0c000 mov  ip, r0          # ip = a[0]
c: 212ffff1e bxcs lr            # don't let return address change ISAs
10: e92d4030 push {r4, r5, lr}   # save r4, r5, return address
Outer Loop:
14: e5bc4004 ldr  r4, [ip, #4]!   # x = a[i] ; increment ip
18: e1a02003 mov  r2, r3          # j = i
1c: e1a0e00c mov  lr, ip         # lr = a[0] (using lr as scratch reg)
Inner Loop:
20: e51e5004 ldr  r5, [lr, #-4]   # r5 = a[j-1]
24: e1550004 cmp  r5, r4          # compare a[j-1] vs. x
28: da000002 ble  38             # if a[j-1] <= a[i], jump to Exit Inner Loop
2c: e2522001 subs r2, r2, #1      # j--
30: e40e5004 str  r5, [lr], #-4   # a[j] = a[j-1]
34: 1afffff9 bne  20             # if j != 0, jump to Inner Loop
Exit Inner Loop:
38: e2833001 add  r3, r3, #1      # i++
3c: e1530001 cmp  r3, r1          # i vs. n
40: e7804102 str  r4, [r0, r2, lsl #2] # a[j] = x
44: 3afffff2 bcc  14             # if i < n, jump to Outer Loop
48: e8bd8030 pop  {r4, r5, pc}   # restore r4, r5, and return address
```

ARM-v32程序

19个指令
76个字节

2.6 典型指令系统结构

第 110 页

```
# MIPS-32 (24 instructions, 96 bytes, or 56 bytes with microMIPS)
# a1 is n, a3 is pointer to a[0], v0 is j, v1 is i, t0 is x
0: 24860004 addiu a2,a0,4 # a2 is pointer to a[i]
4: 24030001 li    v1,1    # i = 1
Outer Loop:
8: 0065102b sltu  v0,v1,a1 # set on i < n
c: 14400003 bnez  v0,1c    # if i<n, jump to Continue Outer Loop
10: 00c03825 move a3,a2    # a3 is pointer to a[j] (slot filled)
14: 03e00008 jr   ra       # return from function
18: 00000000 nop          # branch delay slot unfilled
Continue Outer Loop:
1c: 8cc80000 lw    t0,0(a2) # x = a[i]
20: 00601025 move  v0,v1    # j = i
Inner Loop:
24: 8ce9ffff lw    t1,-4(a3) # t1 = a[j-1]
28: 00000000 nop          # load delay slot unfilled
2c: 0109502a slt   t2,t0,t1 # set a[i] < a[j-1]
30: 11400005 beqz  t2,48    # if a[j-1]<=a[i], jump to Exit Inner Loop
34: 00000000 nop          # branch delay slot unfilled
38: 2442ffff addiu v0,v0,-1 # j--
3c: ace90000 sw    t1,0(a3) # a[j] = a[j-1]
40: 1440fff8 bnez  v0,24    # if j != 0, jump to Inner Loop
44: 24e7ffff addiu a3,a3,-4 # decr. a2 to point to a[j] (slot filled)
Exit Inner Loop:
48: 00021080 sll   v0,v0,0x2 #
4c: 00821021 addu  v0,a0,v0  # v0 now byte address oi a[j]
50: ac480000 sw    t0,0(v0)  # a[j] = x
54: 24630001 addiu v1,v1,1   # i++
58: 1000ffeb b     8         # jump to Outer Loop
5c: 24c60004 addiu a2,a2,4   # incr. a2 to point to a[i] (slot filled)
```

MIPS-v32程序

24个指令
96个字节

2.6 典型指令系统结构

第 111 页

```
# x86-32 (20 instructions, 45 bytes)
# eax is j, ecx is x, edx is i
# pointer to a[0] is in memory at address esp+0xc, n is in memory at esp+0x10
0: 56          push esi          # save esi on stack (esi needed below)
1: 53          push ebx          # save ebx on stack (ebx needed below)
2: ba 01 00 00 00 mov  edx,0x1    # i = 1
7: 8b 4c 24 0c  mov  ecx,[esp+0xc] # ecx is pointer to a[0]

Outer Loop:
b: 3b 54 24 10  cmp  edx,[esp+0x10] # compare i vs. n
f: 73 19        jae  2a <Exit Loop> # if i >= n, jump to Exit Outer Loop
11: 8b 1c 91     mov  ebx,[ecx+edx*4] # x = a[i]
14: 89 d0        mov  eax,edx      # j = i

Inner Loop:
16: 8b 74 81 fc  mov  esi,[ecx+eax*4-0x4] # esi = a[j-1]
1a: 39 de        cmp  esi,ebx        # compare a[j-1] vs. x
1c: 7e 06        jle  24 <Exit Loop>    # if a[j-1] <= a[i], jump Exit Inner Loop
1e: 89 34 81     mov  [ecx+eax*4],esi # a[j] = a[j-1]
21: 48           dec  eax              # j--
22: 75 f2        jne  16 <Inner Loop>    # if j != 0, jump to Inner Loop

Exit Inner Loop:
24: 89 1c 81     mov  [ecx+eax*4],ebx # a[j] = x
27: 42           inc  edx              # i++
28: eb e1        jmp  b <Outer Loop>    # jump to Outer Loop

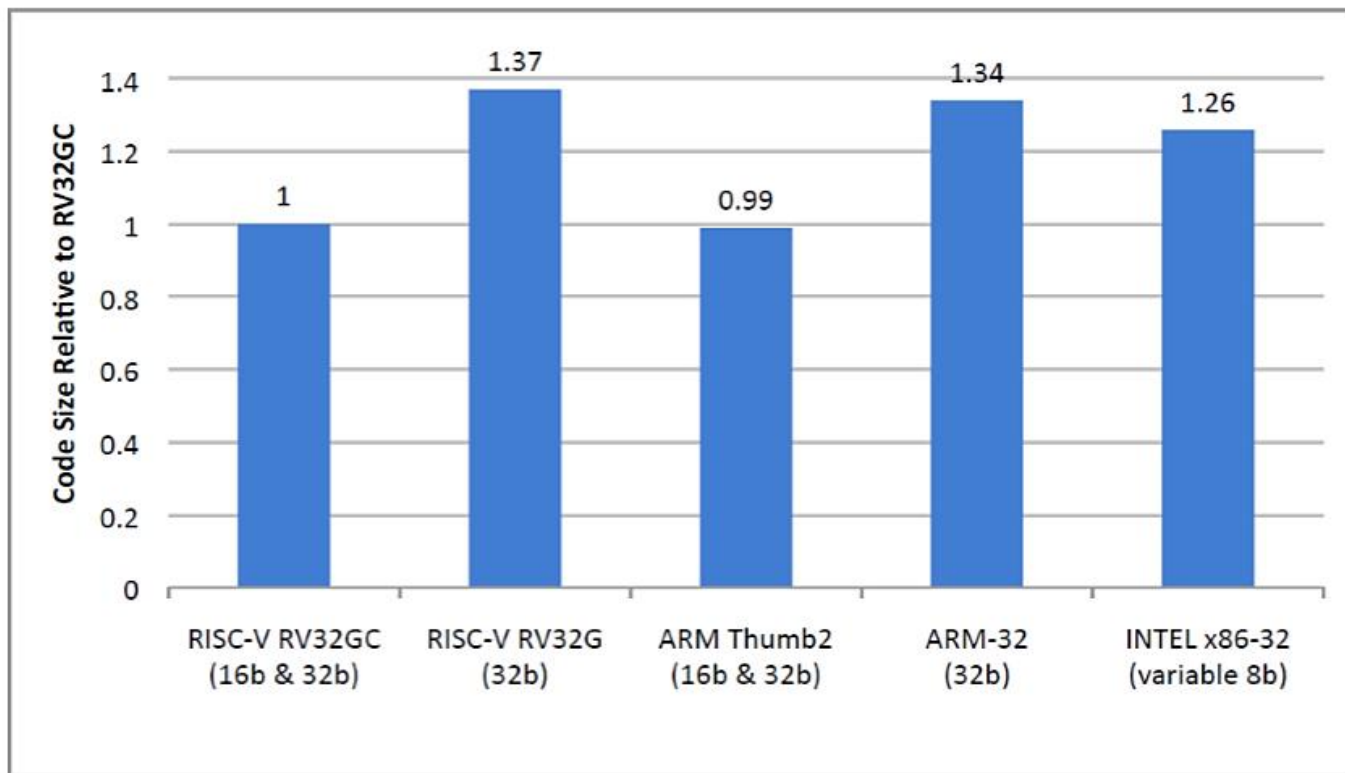
Exit Outer Loop:
2a: 5b          pop  ebx          # restore old value of ebx from stack
2b: 5e          pop  esi          # restore old value of esi from stack
2c: c3          ret           # return from function
```

X86-v32程序

20个指令
45个字节

2.6 典型指令系统结构

第 112 页



RISC-V指令集的代码密度相对优化

指令系统	ARM-v32	MIPS-v32	X86-v32	RV32I
指令数	19	24	20	19
字节数	76	96	45	76

1. 指令系统分类：存储单元类型，堆栈，累加器，通用寄存器组，寄存器-寄存器结构
2. 数据类型和表示：数据类型，数据表示，确定原则，操作数大小，访问频率
3. 寻址方式：整数边界，立即数寻址，直接寻址，寄存器寻址，寄存器间接寻址，寄存器相对寻址，基址变址寻址，相对基址变址寻址
4. 数据操作：数据传送类指令，算术运算指令，逻辑运算指令，程序控制指令
5. 指令系统的设计与优化：完整性，规整性，正交性，高效率，兼容性，控制指令，哈夫曼编码，扩展操作码，等长扩展码，定长操作码
6. 指令系统的发展：CISC、RISC
7. 典型指令系统结构：RISC-V指令集、代码密度

选择题

第 114 页

1、不需要编址的数据存储空间是（ **D** ）

A. CPU中的通用寄存器 B. 主存储器 C. I/O接口中的寄存器 D. 堆栈

2、信息按整数边界存储的主要优点是（ **A** ）

A. 访问速度快 B. 节约主存单元 C. 指令字的规整化 D. 指令的优化

3、操作码优化的主要目的是（ **D** ）

A. 缩短指令字长 B. 减少程序总位数 C. 增加指令字表示的信息 D. A、B和C

4、平均码长最短的编码是（ **B** ）

A. 定长编码 B. 哈夫曼编码 C. 扩展编码 D. 需要根据编码使用的频度计算平均码长后确定

选择题

第 115 页

5、2-4扩展编码最多可以得到的码点数是 (**D**)

A. 6 B. 7 C. 10 D. 13

说明：取1/12等长扩展编码，例如2位的只取00，其他四位的码取01XX；10XX；11XX，XX都是00,01,10,11四种取法，共有13种

2-4-6扩展编码方法的最短码长是 **2** 位，最长码长是 **6** 位，最多可编码的码点数 **46**。

说明：取1/1/44等长扩展编码，例如2位的码只取00，4位的码取0100，其他0101XX、0110XX、0111XX、1000XX、1001XX、1010XX、1011XX、1100XX、1101XX、1110XX、1111XX都是00,01,10,11四种取法，共有44种。

选择题

第 116 页

6、面向目标程序优化的思想是 (C)

- A. 通过使用频度分析来改进指令系统
- B. 增强功能复合指令代替原来的软件实现
- C. A和B
- D. 面向编译系统改进指令系统

7、RISC执行程序的速度比CISC要快的原因是 (C)

- A. RISC的指令系统中指令条数较少
- B. 程序在RISC上编译生成的目标程序较短
- C. RISC的指令平均执行周期
- D. RISC只允许load和store指令访存

8、RISC采用寄存器窗口重叠技术，从而大大减少了 (B)

- A. 绝大多数指令的执行时间
- B. 程序调用引起的访存次数
- C. 目标程序的指令条数
- D. CPU访存的访问周期

1、设某台计算机有9条指令，各指令的使用频度分别为：

I_1 : 52% I_2 : 14% I_3 : 12% I_4 : 7% I_5 : 6%
 I_6 : 5% I_7 : 2% I_8 : 1% I_9 : 1%

分别用哈夫曼编码和2-4-6扩展编码为其操作码编码，并分别计算平均码长。

解：

指令	使用频度 P_i	哈夫曼编码	2-4-6编码
I_1	0.52	0	00
I_2	0.14	100	01
I_3	0.12	110	10
I_4	0.07	1010	1100
I_5	0.06	1011	1101
I_6	0.05	1110	1110
I_7	0.02	11110	111100
I_8	0.01	111110	111101
I_9	0.01	111111	111110

哈夫曼编码的平均码长：

$$L = \sum_{i=1}^9 p_i l_i = 2.06 \text{位}$$

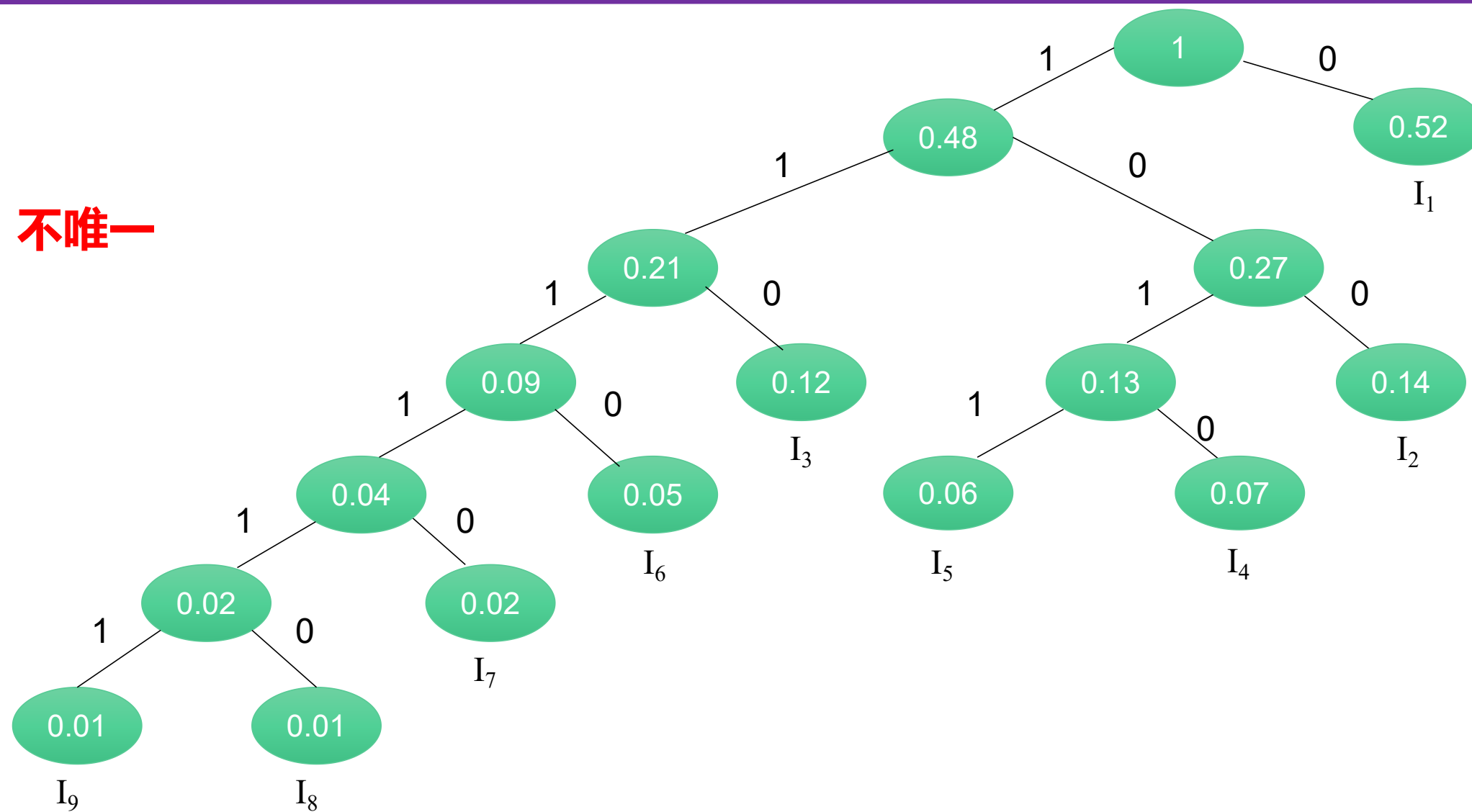
2-4-6扩展编码的平均码长：

$$L = \sum_{i=1}^9 p_i l_i = 2.52 \text{位}$$

应用题

第 118 页

不唯一



2、设某台处理机的各条指令使用频度如下表所示。

指令	使用频度	指令	使用频度	指令	使用频度
ADD	43%	JOM	6%	CIL	2%
SUB	13%	STO	5%	CLA	22%
JMP	7%	SHR	1%	STP	1%

请分别设计这9条指令操作码的哈夫曼编码、3/3/3等长扩展编码和2/7扩展编码，并计算这3种编码的平均码长。

应用题

指令	使用频度	哈夫曼编码1	哈夫曼编码1	3/3/3扩展编码	2/7扩展编码
ADD	0.43	0	0	00	00
CLA	0.22	10	100	01	01
SUB	0.13	110	101	10	1000
JMP	0.07	11100	1100	1100	1001
JOM	0.06	11101	1101	1101	1010
STO	0.05	11110	1110	1110	1011
CIL	0.02	111110	11110	111100	1100
SHR	0.01	1111110	111110	111101	1101
STP	0.01	1111111	111111	111110	1110
平均码长	/	2.42	2.42	2.52	2.70

第3章 流水线技术